

Rewriting Ontology-Mediated Property Graph Queries into GQL

Bianca Löhnert
bianca.loehnert@plus.ac.at
University of Salzburg
Austria

Cem Okulmus
cem.okulmus@upb.de
Paderborn University
Germany

Nikolaus Augsten
nikolaus.augsten@plus.ac.at
University of Salzburg
Austria

Magdalena Ortiz
magdalena.ortiz@tuwien.ac.at
TU Wien
Austria

Abstract

Ontology-based data access is intended for graph data, but practical support remains limited to SQL-like languages lacking the navigational and path-matching features fundamental to graph querying. Theoretical algorithms for navigational queries have long been available. Still, they have never been implemented, largely because practical graph query languages fell short of their theoretical counterparts and lacked the expressive power needed to support rewriting common ontology languages.

The recent standardisation efforts around GQL and SQL/PGQ finally allow us to overcome this barrier and present a practical query rewriting technique for ontology-mediated navigational graph queries. Ontologies are written in a DL-Lite variant tuned to property graphs, and the queries in a GQL fragment with nested two-way regular path queries and which can be evaluated in Cypher. Preliminary experiments with our proof-of-concept prototype suggest that querying graph data with ontological knowledge may finally be within reach.

CCS Concepts

• **Theory of computation** → **Description logics**; • **Information systems** → **Query languages**; *Graph-based database models*.

Keywords

ontology-based data access, graph query languages, ontology-mediated query answering, description logics

ACM Reference Format:

Bianca Löhnert, Nikolaus Augsten, Cem Okulmus, and Magdalena Ortiz. 2026. Rewriting Ontology-Mediated Property Graph Queries into GQL. In *Proceedings of International Conference on Information and Knowledge Management (CIKM)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3799682.3841030>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM, Rome, IT

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/10.1145/3799682.3841030>

1 Introduction

A core aim of the *ontology-based data access* (OBDA) framework, in addition to the virtual integration of data sources, is to extend existing data with domain-specific knowledge without the need to materialise all implied facts; the framework still enables access to all consequences when answering user queries. The key question at the core of OBDA is the following: how to capture all the semantic consequences of a query in the presence of an ontology, where the query may be specified over a possibly *extended* signature with terms from both the data and the ontology. Such a pair of query and ontology is often called an *ontology-mediated query* (OMQ) and the task of producing all consequences is called OMQ answering (OMQA). The main technique for OMQA is called *query rewriting*: the query is rewritten into a new query that uses only the *reduced* signature present in the data, incorporating the knowledge in the ontology into the query itself. The rewritten query can then be evaluated using a conventional database system that does not support ontologies, while still making effective use of the ontology to retrieve implicit answers.

While commercial OBDA systems already exist for relational databases [40], corresponding solutions for graph databases—the focus of this paper—are still missing. Instead of the relational model, graph databases focus on the labelled property graph (LPG) model, which allows for nodes and binary, directed edges between nodes. Both nodes and edges can be assigned multiple *labels* to classify them, and they can be associated with data values via so-called *properties*. The increasing popularity of graph databases has recently culminated in the ISO standardisation of two new graph query languages: GQL and SQL/PGQ. The key distinguishing feature of these query languages is their *navigational* capability, which allows them to match arbitrarily long paths in the data. Both GQL and SQL/PGQ share the same navigational core [15], which includes the standard navigational query languages: (two-way) regular path queries ((2)RPQs), nested 2RPQs (N2RPQs), and their conjunctive variants (C2RPQs resp. CN2RPQs).

The extensive study of navigational queries in the context of OMQA over the past decade has led to tight complexity results for most description logics, ranging from lightweight to highly expressive, as well as for all standard navigational query languages (2RPQs, N2RPQs, C2RPQs, and CN2RPQs) [8, 10, 12, 17, 32, 33, 37]. However, these studies focused on the boundaries of decidability and computational complexity. The proposed algorithms are so unamenable to implementation that, more than a decade later, not a single practical implementation has been proposed. We differ from

these early theoretical works, by looking to develop practical techniques for pure rewritings of navigational queries (i.e., the ontology reasoning is compiled entirely into the rewritten query, which can be evaluated with standard query engines without any data preprocessing) that can pave the way towards practical implementations.

There were two significant roadblocks that discouraged the development of practical techniques for OMQA over graph databases. First, practical languages imposed ad-hoc restrictions on navigational languages that rendered them unsuitable for this setting, for example, by providing only partial support for RPQs and by failing to support the basic homomorphism semantics that is natural in the OMQA context. Second, it has been shown that even plain RPQs in the presence of the simplest DL-Lite ontologies cannot be rewritten into C2RPQs [28]. Instead, such rewritings require *nested* RPQs, which existing technologies did not support. The arrival of GQL and SQL/PGQ has removed both obstacles at once: the standards include CN2RPQs and basic homomorphism semantics [20, 21], and practical graph query languages are now rapidly being updated to support these features [31]. This has finally made practical OMQA over graph databases feasible.

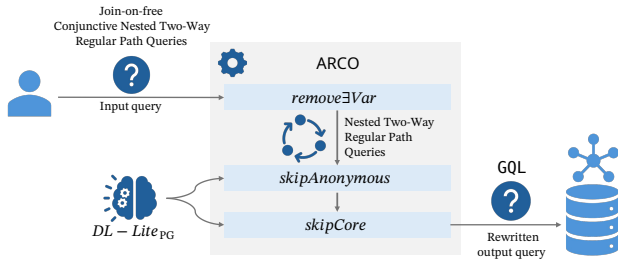


Figure 1: High-level overview of the rewriting algorithm.

Seizing this opportunity, we propose the first practical query rewriting technique for a navigational query language that contains RPQs. in the presence of DL-Lite ontologies. Figure 1 provides an overview of the overall architecture, which comprises a DL-Lite ontology expressed, for example, in the Web Ontology Language (OWL) [39]; our rewriting algorithm “Automata-based Rewriting of CN2RPQs under Ontologies” or simply ARCO for short; and a property graph database with support for GQL, such as Neo4j,¹ Google Spanner,² or Oracle Database (via SQL/PGQ).³ The user poses a query over the extended signature, i.e., the query may use terms from both the ontology and the underlying database. Our rewriting algorithm compiles the user query into a semantically equivalent rewritten query that incorporates the consequences of ontology reasoning directly into the query expression. The rewritten query is subsequently evaluated by the graph database system. Although the database remains agnostic to the ontology, the answer to the rewritten query is complete with respect to the ontology semantics.

In terms of query language, our approach fully covers 2RPQs and N2RPQs in the presence of DL-Lite ontologies. We also consider conjunctive N2RPQs; to ensure the algorithm remains practical

and useful, we impose a restriction called *join-on-free*, whereby non-answer variables cannot be shared by multiple atoms. While techniques for rewriting full C2RPQs are known [19], this fragment serves as a useful starting point for our endeavour. We also support property data values, a central feature of the labelled property graph model that is often disregarded in the OMQA literature. Property tests are also allowed along navigational paths.

On the ontology side, we define a variant of DL-Lite that can use property value tests on the left-hand-side of axioms, which enables the creation of concepts and roles based on these values.

In summary, we make the following contributions:

- (1) We present the first practical algorithm to rewrite navigational ontology-mediated queries with unrestricted regular path expressions, and with DL-Lite ontologies, into graph queries that can be evaluated with off-the-shelf query engines for graph data adopting the property graph data model. The queries can access the data values of properties, also while navigating paths.
- (2) We leverage the power of graph queries with *nested path navigation* to support the interaction between the regular path expressions in the given queries on the one hand, and the ontology axioms on the other. To our knowledge, this is the first algorithm for a navigational query language that fully supports property paths, which produces a *pure rewriting* that can be evaluated directly over the data, with the ontology consequences entirely rewritten into the query.
- (3) We demonstrate the practical utility of our technique by implementing a prototype. The system takes as input an ontology in OWL format together with navigational queries, rewrites them into the graph query language Cypher, and evaluates them on a database in Neo4j.

This paper is structured as follows. In Section 2 we present the key terminology of our setting and also introduce our novel description logic to express tests over property values. In Section 3 we define procedures to rewrite N2RPEs in the presence of *DL-Lite_{PG}* ontologies. Then, in Section 4 we use these procedures to present our rewriting algorithm on join-on-free CN2RPQs. In Section 5 we present our proof-of-concept implementation and an experimental evaluation that aims to show its performance. In Section 6 we summarise our results and highlight future work.

Related Work. ⁴ Early work by Di Martino et al. [16, 17] leverages the recursion in regular path queries to rewrite a fragment of $\mathcal{EL}$, the lightweight description logic underlying the OWL 2 EL profile. They use navigational queries as target language for query rewriting, but their source languages are only instance queries and conjunctive queries; we support navigational source queries. The work is primarily theoretical and not aimed at implementation. It is therefore more closely related to the previously discussed theoretical studies of navigational queries in the context of OMQA [8, 10, 12] than to this paper. Aiming at practical implementation, Dragovic et al. [18] considered *DL-Lite* ontologies and a restricted fragment of C2RPQs that can be rewritten into unions of C2RPQs. However, the approach does not cover full RPQs. Like Di Martino et al. [16], Löhnert et al. [28] support a fragment of $\mathcal{EL}$ as an ontology; their

¹<https://neo4j.com/>

²<https://cloud.google.com/spanner>

³<https://www.oracle.com/database/>

⁴This work is a corrected version of the preliminary algorithm presented in [27].

regular path expressions are restricted to ensure rewritability into C2RPQs, i.e., to avoid the need for nested queries. Query rewriting in the presence of data tests has been explored previously, such as the work by Savković and Calvanese [35], which considered the problem of OMQ answering over an extension of DL-lite with expressions over concrete domains called datatype expressions, and the ability to express hierarchies over them. Baader et al. [5] extended this work by an even more expressive language, where the datatype expressions over the concrete domain can use n -ary predicates instead of just unary ones. Our restricted use of data values does not require reasoning over the data domains, and we can simply rely on the query engine to evaluate the data atoms.

2 Ontology Mediated Querying of Property Graphs

Data model. We are interested in querying data given as finite *property graphs* [1, 11, 22].

We assume disjoint, countably infinite sets $\mathbf{C}$, $\mathbf{R}$, $\mathbf{K}$ and $\mathbf{N}$ of *concept names*, *role names*, *PROPERTY KEYS* and *individuals*, respectively. In the examples, we colour code the first three, as shown, for convenience. We also assume a *concrete domain* $(\mathbf{D}, \mathbf{P}^{\mathbf{D}})$, with $\mathbf{D}$ a set of *values* and $\mathbf{P}^{\mathbf{D}}$ a set of binary predicates over $\mathbf{D}$. For example, $(\mathbf{D}, \mathbf{P}^{\mathbf{D}})$ could be the integers with the usual $=, \leq, \geq$ predicates.

Definition 1 (Property Graphs). A *property graph* (PG) $\mathcal{A}$ has the form $(N, E, \text{label}, \text{prop})$, where:

- N is a non-empty set of *nodes*;
- E is the set of *edges*, where each edge is a triple (r, n, n') with $r \in \mathbf{R}$ and $n, n' \in N$; we may write such an edge in the form $r(n, n')$ and call it an *r-edge*;
- *label* is a total function $N \rightarrow 2^{\mathbf{C}}$;
- *prop* is a partial function $(N \cup E) \times \mathbf{K} \rightarrow \mathbf{D}$ mapping pairs (u, p) with $u \in (N \cup E)$ and $p \in \mathbf{K}$ to a value in $\mathbf{D}$.

If $N' \subseteq N$ and it is finite, we may call $\mathcal{A}$ an *ABox*. We say that $\mathcal{A}' = (N', E', \text{label}', \text{prop}')$ is a *subgraph* of $\mathcal{A}$ if $N' \subseteq N$, $E' \subseteq E$, $\text{label}'(n) = \text{label}(n)$ for all $n \in N'$, and $\text{prop}'(u, p) = \text{prop}(u, p)$ for all $u \in N' \cup E'$, $p \in \mathbf{K}$.

Note that we use the same name of property keys for both nodes and edges, as usually done in PGs [11, 22]. We differ slightly from the usual definition of PGs, by allowing only a single edge for each role between pairs of nodes. This is puts our data model in between the usual DL setting and what real-world PG database systems like Neo4j [22] support, where multiple edges with the same role between the same pair of nodes are allowed

Example 2. Figure 2 shows profile information from a social network (such as LinkedIn), represented as a PG.

Ontology language. In ontology mediated query answering, an ontology captures domain knowledge and it allows us to infer implicit facts from the given data at query time. Here, the ontology language is called *DL-Lite_{PG}*. It extends the well-known DL-Lite [13] with tests over property values, which can be used in the left-hand-side both concept and role inclusions.

Definition 3 (DL-Lite_{PG}). The set of *roles* is defined as $\bar{\mathbf{R}} = \mathbf{R} \cup \{r^- \mid r \in \mathbf{R}\}$, where r^- is called the *inverse of r* and the set of *data tests* as $\mathbf{T}^{\mathbf{D}} = \{p \odot v \mid p \in \mathbf{K}, \odot \in \mathbf{P}^{\mathbf{D}}, v \in \mathbf{D}\}$.

We use the following syntax, where $A \in \mathbf{C}$, $r \in \bar{\mathbf{R}}$ and $T \subseteq \mathbf{T}^{\mathbf{D}}$.

$$\begin{aligned} B &:= \top \mid A \mid \exists r. \top \mid T && \text{(basic concepts, data-test concepts)} \\ E &:= r \mid r^- && \text{(basic roles)} \\ C &:= B \mid \neg B && \text{(general concept)} \\ F &:= E \mid T && \text{(roles, data-test roles)} \\ R &:= E \mid \neg E && \text{(general role)} \end{aligned}$$

The symbol $\top$, called ‘top concept’, refers to the set of all nodes of the domain in question. A *concept inclusion* (CI) has the form $B \sqsubseteq C$ and a *role inclusion* (RI) the form $F \sqsubseteq R$. A TBox is a finite set of CIs and RIs. We use $\sqsubseteq_{\mathcal{T}}^*$ to denote the reflexive transitive closure of $\{(r, t) \mid r \sqsubseteq t \in \mathcal{T}\}$ and call r a *subrole of t* (in $\mathcal{T}$) if $r \sqsubseteq_{\mathcal{T}}^* t$.

The semantics of concepts, roles and data sets is defined with respect to a given PG $\mathcal{A}$. In the description logic (DL) literature, this is usually defined by a so-called interpretation of the form $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$, with $\Delta^{\mathcal{I}}$ a non-empty set called the *abstract domain* and $\cdot^{\mathcal{I}}$ as the interpretation function. In this paper, we consider a given PG to be an interpretation in the DL sense. Hence, given a PG $\mathcal{A}$, by slight abuse of notation, we may also use it as an interpretation function that maps concepts, roles and data sets to, resp., sets of nodes, pairs of nodes or a union of the two. This function is given in Table 1. In the sequel, when the PG $\mathcal{A} = (N, E, \text{label}, \text{prop})$ is clear from context, we shall refer to it as “interpretation” and, for example, refer to its set of nodes N as its domain $\Delta^{\mathcal{A}}$.

As for the rest of the semantics, the satisfaction of CIs and RIs is as expected: $\mathcal{I} \models C \sqsubseteq D$ iff $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$, and $\mathcal{I} \models E \sqsubseteq F$ iff $E^{\mathcal{I}} \subseteq F^{\mathcal{I}}$. We write $\mathcal{I} \models \mathcal{T}$ iff $\mathcal{I} \models C \sqsubseteq D$ for every $C \sqsubseteq D \in \mathcal{T}$. We write $\mathcal{T} \models C \sqsubseteq D$ iff for every PG $\mathcal{I}$ with $\mathcal{I} \models \mathcal{T}$ we also have $\mathcal{I} \models C \sqsubseteq D$.

For readability, in the examples we use CIs of the form $A \sqsubseteq \exists r.B$, which are not strictly in *DL-Lite_{PG}*. These are just syntactic sugar: we can write $A \sqsubseteq \exists r.B$ in *DL-Lite_{PG}* using a fresh role name r_B and three axioms: $A \sqsubseteq \exists r_B. \top$, $\exists r_B. \top \sqsubseteq B$, $r_B \sqsubseteq r$.

Example 4. The following *DL-Lite_{PG}* TBox provides knowledge that can be applied to the social network graph of Example 2:

$$\mathcal{T} = \left\{ \begin{array}{l} \{\text{BORN} \geq 1997?, \text{BORN} \leq 2012?\} \sqsubseteq \text{GenZ}, \\ \exists \text{emp}^-. \top \sqsubseteq \text{Employed}, \{\text{TIME} \geq (2025-01-01)?\} \sqsubseteq \text{Recent}, \\ \text{Employed} \sqsubseteq \neg \text{Unemployed}, \text{Opole} \sqsubseteq \text{Poland}, \exists \text{ann}. \top \sqsubseteq \text{Hiring} \\ \text{TechCompany} \sqsubseteq \exists \text{emp}. \text{Engineer}, \text{friend} \sqsubseteq \text{friend}^- \end{array} \right\}$$

To briefly explain a few of the axioms, we see that the first axiom in $\mathcal{T}$ defines the new concept *GenZ*, which applies to anyone born between the years 1997 and 2012. The second axiom says that someone who has an employer is employed, and the fourth that no one can be both employed and unemployed.

For the problem of checking whether a given PG is consistent with an *DL-Lite_{PG}* TBox, we refer to the work of Artale et al. [4]; their algorithms can be easily tuned to account for our restricted use of property values.

Query Language. We study *conjunctive nested two-way regular path queries* (CN2RPQs), the extension of C2RPQs, the navigational query language for graphs that has received most attention in OMQA. We enhance CN2RPQs by *data tests* similar to data tests for navigational conjunctive queries in [18] to query for property values, assuming that the predicates in $\mathbf{P}^{\mathbf{D}}$ can be realized in GQL and

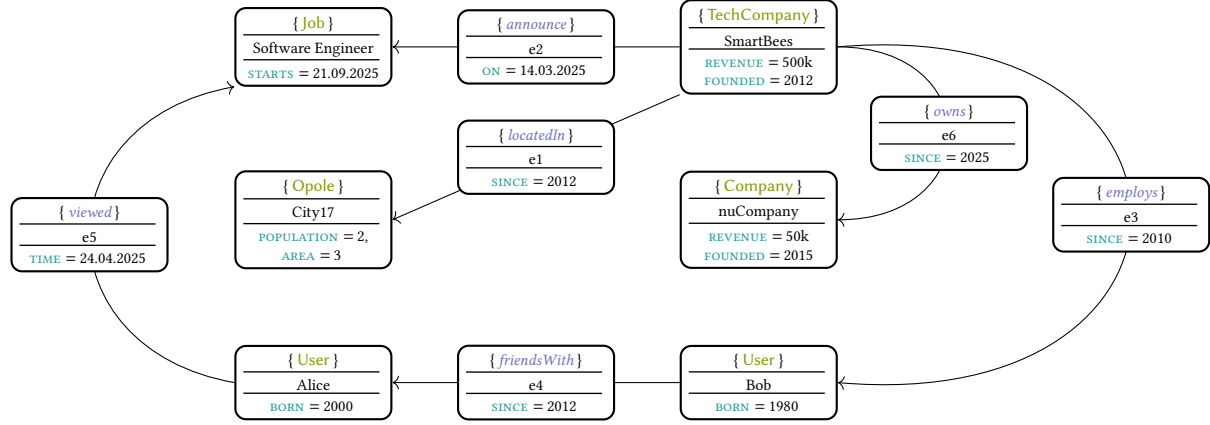


Figure 2: Property Graph of Example 2

Table 1: Semantics of $DL\text{-}Lite_{PG}$ concepts, roles and data tests given $\mathcal{A} = (N, E, \text{label}, \text{prop})$.

C	$C^{\mathcal{A}} \subseteq N$	F	$F^{\mathcal{A}} \subseteq N \times N$	T	$T^{\mathcal{A}} \subseteq (N \times N) \cup N$
$\top$	N	r	$\{(n, m) \mid r(n, m) \in E\}$	$p \odot v?$	$\{n \in N \mid v' \odot v \text{ where } \text{prop}(n, p) = v'\} \cup$
A	$\{n \in N \mid A \in \text{label}(n)\}$	r^-	$\{(m, n) \mid r(n, m) \in E\}$		$\{(n, m) \mid v' \odot v \text{ where } \exists r \in \mathbf{R}. (\text{prop}(r(n, m), p) = v' \text{ and } r(n, m) \in E)\}$
$\exists r. \top$	$\{n \mid r(n, m) \in E\}$			$\{T_1, \dots, T_n\}$	$\{T_1^{\mathcal{A}} \cap \dots \cap T_n^{\mathcal{A}}\}$
$\neg C$	$N \setminus C^{\mathcal{A}}$				

Cypher. To represent CN2RPQs we rely on *nested nondeterministic finite automaton* (n-NFA), following generally the notions from [8], with some simplifications.

We first define nested two-way regular path expressions using NFAs, and then define conjunctive nested two-way regular path queries based on them. We assume a given alphabet Σ , and to define nested automata, we extend it by allowing n-NFAs as symbols.

Definition 5. Let A^0 be the set of all NFAs over a given alphabet Σ . For $k > 0$, the set A^k of *nested NFA* (n-NFA) over Σ of nesting depth k contains all automata over $\Sigma^k = \Sigma \cup \{\langle \alpha \rangle \mid \alpha \in A^i, 1 \leq i < k\}$.

We omit the nesting depth of an n-NFA when it is irrelevant. We are interested in n-NFAs that use the alphabet of symbols and tests that may occur in our PGs.

Definition 6. A *nested two-way regular path expression* (N2RPE) is an n-NFA over the specific alphabet

$$\Sigma_{PG} = 2^{\bar{\mathbf{R}} \cup \mathbf{T}^D} \cup \mathbf{T}^D \cup \{C? \mid C \in \mathbf{C}\}.$$

Σ_{PG} consists of three kinds of symbols: 1) sets of (possibly inverted) role labels and data tests, which allow the expression to perform a series of data tests while traversing an edge that has all the role labels in the test; 2) individual data tests, which are checked against the properties of a node and lastly, 3) concept tests, which check whether the current node is part of the required concept.

Sets of binary (data and role) tests allow us to do several tests while traversing one edge. In contrast, sets of unary (data and concept) tests do not need to be performed simultaneously; they can be tested equivalently by consecutive transitions.

We can now use N2RPEs as a query language for PGs. We define the semantics of a N2RPE α via paths that match α , possibly with outgoing paths to match the nested automata.

Definition 7 (Run of an N2RPEs). Let $\alpha_0 = \langle Q_0, \Sigma_{PG}, S_0, \delta_0, F_0 \rangle$ be an N2PRE of nesting depth k and let $\alpha_1, \dots, \alpha_m$ be the N2PREs of nesting depth $< k$ mentioned in the transitions of α_0 . We let $\alpha_i = \langle Q_i, \Sigma_{PG}, S_i, \delta_i, F_i \rangle$ for each α_i . Then, a *run* for α on $\mathcal{I}$ is a finite node-labelled tree (T, l) such that every node is labelled with an element from $\Delta^{\mathcal{I}} \times \bigcup_i Q_i$. Furthermore, for the root r_T of T we have that $l(r_T) = (n_r, s_r)$, where $s_r \in S_0$ and for every leaf node $e \in N(T)$, it holds that $l(e) = (n, s_n)$, with $s_n \in F_i$ for some i and there exists exactly one leaf n' such that $l(e') = (n', s')$, with $s' \in F_0$, called the *accepting node*. For each non-leaf node $v \in N(T)$ having label $l(v) = (n_v, s_v)$ with $s_v \in Q_i$, one of the following holds:

- v has a unique child v_c with $l(v_c) = (n_c, s_c)$, and there exists $(s_v, \sigma, s_c) \in \delta_i$, such that one of the following holds:
 - If $\sigma \in 2^{\bar{\mathbf{R}} \cup \mathbf{T}^D}$ then:
 - (i) $(n_v, n_c) \in (p \odot v?)^{\mathcal{I}}$ for each data test $p \odot v? \in \sigma^{\mathcal{I}}$, and
 - (ii) $(n_v, n_c) \in r^{\mathcal{I}}$ for each $r \in \sigma^{\mathcal{I}} \cap \mathbf{R}$;
 - if $\sigma = C?$, then $n_c = n_v$ and $n_v \in C^{\mathcal{I}}$, and
 - if $\sigma = p \odot v?$, then $n_v = n_c$ and $n_v \in (p \odot v?)^{\mathcal{I}}$.
- v has exactly two children v_{c_1} and v_{c_2} with $l(v_{c_1}) = (n_v, s_{c_1})$ and $l(v_{c_2}) = (n_v, s_{c_2})$, with s_{c_2} the initial state of α_j , and there exists a transition $(s_v, \langle \alpha_j \rangle, s_{c_1}) \in \delta_i$.

If the root r_T is labelled (n_r, s_0) and the accepting node u is labelled (n_u, s_u) with $s_0 \in S_0$ and $s_u \in F_0$, then (T, l) is called an (n_r, n_u) -run, and if such a run exists, we write $(n_r, n_u) \in \alpha_0^{\mathcal{I}}$.

N2RPEs are a very rich query language in their own right, but we get even more flexible querying if we join N2RPEs using variables.

Definition 8. A conjunctive N2RPQ (CN2RPQ) $q(\vec{x})$ is a conjunction of atoms $\alpha_1(x_1, y_1) \wedge \dots \wedge \alpha_i(x_i, y_i) \wedge \dots \wedge \alpha_n(x_n, y_n)$ with each α_i an N2RPE and $\vec{x} \subseteq \bigcup_i \{x_i, y_i\}$ is a tuple of answer variables. We call a CN2RPQ Boolean if $\vec{x}$ is empty.

The set of join variables $\text{Join}(q(\vec{x}))$ of a CN2RPQ $q(\vec{x})$ is the set of those variables that occur in two different atoms of $q(\vec{x})$. If $\text{Join}(q(\vec{x})) \subseteq \vec{x}$ for a CN2RPQ that is not Boolean, then we call $q(\vec{x})$ a join-on-free CN2RPQ.

We sometimes abbreviate atoms of the form $\langle \alpha \rangle(x, x)$ as $\langle \alpha \rangle(x)$, and similarly, $A?(x)$ as $A(x)$.

We assume that CN2RPQs are connected, that is, the graph whose vertices are the variables and that has an edge between two variables if they occur in the same atom is a connected graph. Disconnected queries can be answered as separate queries and then their answers intersected. Note that, under this assumption, every atom in a join-on-free CN2RPQ has at least one answer variable.

Definition 9 (Semantics of CN2RPQs). Consider a CN2RPQs $q(\vec{x})$ and a PG $\mathcal{A} = (N, E, \text{label}, \text{prop})$. A match μ for $q(\vec{x})$ in $\mathcal{A}$ is a mapping from the variables in q to nodes in N such that $(\mu(x), \mu(y)) \in \alpha^{\mathcal{A}}$ for every atom $\alpha(x, y)$ occurring as a conjunct in $q(\vec{x})$. The tuple $\mu(\vec{x})$ is then called an answer for $q(\vec{x})$ in $\mathcal{A}$.

Example 10. To illustrate our query language, we use the schema introduced in Example 4 and present four example queries, with informal explanations of their semantics following below.

$q_1(x, y) := \text{friendsWith}^* \cdot \text{employs}^-(x, y),$
 $\langle \text{locatedIn} \cdot \text{Poland?} \rangle \cdot \text{Hiring?} \cdot \text{TechCompany?}(y, z)$
 $q_2(x, y) := \{ \text{employs}^-, \text{SINCE} \leq 2023? \} \cdot \text{friendsWith}^* \cdot \text{BORN} \geq 2000?(x, y),$
 $\langle \{ \text{viewed}, \text{TIME} \geq 01.01.2024? \} \rangle \cdot \text{friendsWith}(z, y)$
 $q_3(x, y) := \text{FOUNDED} \leq 2003? \cdot \text{REVENUE} \geq 10^5?.$
 $\langle \{ \text{owns}, \text{SINCE} \geq 2020? \} \rangle \cdot \langle \text{locatedIn} \cdot \text{Poland?} \rangle^*(x, y)$
 $q_4(x, y) := \text{Company?} \cdot \text{employs} \cdot \text{friendsWith} \cdot \text{GenZ?}(x, y),$
 $\text{viewed} \cdot \text{Job?} \cdot \text{announce}^-(y, x)$

The query $q_1(x, y)$ retrieves all technical companies located in Poland that are currently hiring and that have an employee who is a friend of friends of the user. In query $q_2(x, y)$ a recruiter might be interested in friends (and friends of friends) born after 2000 of an employee, who has been working for the company for over three years. In query $q_3(x, y)$, we look for companies that were founded before 2003, with a revenue of 10^5 or more, and want to find all Polish companies that they own since 2020, and also look for all companies that these companies own, and so on. In query $q_4(x, y)$, we look for all companies that employ workers that are friends with some user of GenZ, and return the pairs of company and users, when the user also viewed a job offer by the company that the friend is currently employed.

Following the OMQA literature, we use the *homomorphism* or *walk* semantics for path queries [2], and in the presence of a TBox, adopt the *certain answer* semantics.

Definition 11. Consider an PG $\mathcal{A}$ and a TBox $\mathcal{T}$. A tuple $\vec{a}$ of individuals in $\mathcal{A}$ is called a *certain answer* to $q(\vec{x})$ over $(\mathcal{T}, \mathcal{A})$ if it is an answer to $q(\vec{x})$ in $PG(I)$ for every model I of $\mathcal{A}$ and $\mathcal{T}$.

The worst-case complexity of evaluating navigational queries over ontologies is well understood. Unfortunately, when we take the ontology's size and the query into account, it is highly intractable.

THEOREM 12 (THEOREM 5.1 & COROLLARY 6.8 [8]). In DL-Lite, N2RPQ answering is NL-complete in data complexity and Exp-complete in combined complexity. In all description logics containing $\mathcal{EL}$ and contained in Horn-SHIQ, N2RPQ answering is P-complete in data complexity and Exp-complete in combined complexity.

Our aim here is not worst-case optimal complexity, but to develop techniques that may lead to tractable evaluation of nested navigational queries over ontologies in practice.

3 Rewriting N2RPQs

We present a rewriting algorithm for N2RPQs with data tests. In this section and the next, we assume a fixed DL-Lite_{PG} TBox $\mathcal{T}$, and provide an algorithm for rewriting a given input N2RPQ q into an N2RPQ $q_{\mathcal{T}}$ that is sound and complete for all ABoxes that are consistent with $\mathcal{T}$. In the next section, we lift this algorithm to (restricted) conjunctions of N2RPQs.

Our algorithm draws inspiration from and builds on previous techniques for navigational ontology-mediated querying [9, 14, 19, 32], but it focuses on potential implementability.

As usual for ontology-mediated querying, we rely on the fact that for every PG $\mathcal{A}$ that is consistent with $\mathcal{T}$, there is a canonical model $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ that gives exactly the certain answers to all N2RPQs. Below we give the construction for the canonical model.

Definition 13 (Canonical Model). Consider a DL-Lite_{PG} TBox $\mathcal{T}$ and a PG $\mathcal{A}$. Let $\mathcal{I}_0 = \mathcal{A}$. Then, we construct the *canonical model* $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ by exhaustively applying the following rules to $\mathcal{I}_0$.

- (Ch1) If $r \sqsubseteq r' \in \mathcal{T}$, $(v, w) \in r^{\mathcal{I}_i}$ and $(v, w) \notin s^{\mathcal{I}_i}$, then $\mathcal{I}_{i+1}$ is $\mathcal{I}_i$ with $r'^{\mathcal{I}_{i+1}} = r'^{\mathcal{I}_i} \cup \{(v, w)\}$ in case $r \in \mathbf{R}$, otherwise $r'^{\mathcal{I}_{i+1}} = r'^{\mathcal{I}_i} \cup \{(w, v)\}$.
- (Ch2) If $T \sqsubseteq r' \in \mathcal{T}$, $(v, w) \in T^{\mathcal{I}_i}$ and $(v, w) \notin r'^{\mathcal{I}_i}$, then $\mathcal{I}_{i+1}$ is $\mathcal{I}_i$ with $r'^{\mathcal{I}_{i+1}} = r'^{\mathcal{I}_i} \cup \{(v, w)\}$.
- (Ch3) If $A \sqsubseteq B \in \mathcal{T}$, $v \in A^{\mathcal{I}_i}$ and $v \notin B^{\mathcal{I}_i}$, then $\mathcal{I}_{i+1}$ is $\mathcal{I}_i$ with $B^{\mathcal{I}_{i+1}} = B^{\mathcal{I}_i} \cup \{v\}$.
- (Ch4) If $\exists r. T \sqsubseteq B \in \mathcal{T}$, $v \in (\exists r. T)^{\mathcal{I}_i}$ and $v \notin B^{\mathcal{I}_i}$, then $\mathcal{I}_{i+1}$ is $\mathcal{I}_i$ with $B^{\mathcal{I}_{i+1}} = B^{\mathcal{I}_i} \cup \{v\}$.
- (Ch5) If $T \sqsubseteq B \in \mathcal{T}$, $v \in T^{\mathcal{I}_i}$ and $v \notin B^{\mathcal{I}_i}$, then $\mathcal{I}_{i+1}$ is $\mathcal{I}_i$ with $B^{\mathcal{I}_{i+1}} = B^{\mathcal{I}_i} \cup \{v\}$.
- (Ch6) If $A \sqsubseteq \exists r. T \in \mathcal{T}$, $v \in A^{\mathcal{I}_i}$ and $v \notin (\exists r. T)^{\mathcal{I}_i}$, then $\mathcal{I}_{i+1}$ is $\mathcal{I}_i$ with a fresh w in $\Delta^{\mathcal{I}_{i+1}}$. In case $r \in \mathbf{R}$ then $r^{\mathcal{I}_{i+1}} = r^{\mathcal{I}_i} \cup \{(v, w)\}$, otherwise $r^{\mathcal{I}_{i+1}} = r^{\mathcal{I}_i} \cup \{(w, v)\}$.
- (Ch7) If $\exists r. T \sqsubseteq \exists r'. T \in \mathcal{T}$, $v \in (\exists r. T)^{\mathcal{I}_i}$ and $v \notin (\exists s. T)^{\mathcal{I}_i}$, then $\mathcal{I}_{i+1}$ is $\mathcal{I}_i$ with a fresh w in $\Delta^{\mathcal{I}_{i+1}}$. In case $s \in \mathbf{R}$ then $r'^{\mathcal{I}_{i+1}} = r'^{\mathcal{I}_i} \cup \{(v, w)\}$, otherwise $r'^{\mathcal{I}_{i+1}} = r'^{\mathcal{I}_i} \cup \{(w, v)\}$.

Note that the property values in DL-Lite_{PG} have no effect when it comes to the canonical model construction. Since data value tests only occur on the left-hand-side of axioms, in $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ the partial function prop stays as-is and does not assign any new data values, neither to nodes in $\mathcal{A}$ nor to freshly added ones.

For convenience, we may think of $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ as a set of facts: $A(o)$ and $r(o, o')$ assert node and edge labels, plus facts $o.(p = v)$, $r(o, o').(p = v)$ that assert property value assignments, where o, o' are nodes, $A \in \mathbf{C}$, $r \in \mathbf{R}$, $p \in \mathbf{K}$ and $v \in \mathbf{D}$. We call a fact *explicit* if

it is present in $\mathcal{A}$, and *implicit* otherwise. That is, implicit facts are those introduced by the canonical model construction that extends the explicit facts to satisfy the TBox $\mathcal{T}$. All property value assertions are explicit. Note that, as usual for DL-Lite variants, each rule application that adds an implicit fact ξ_ℓ tests for the presence of one fact $\xi_{\ell-1}$, which may be explicit or added by a previous rule application. In this way, every implicit fact is triggered by one explicit fact and a sequence of rule applications.

Note that all explicit facts mention only the individual names in $\mathcal{A}$, but the implicit ones may mention additional nodes introduced to satisfy existential restrictions. We call the latter, that is, the nodes in $\mathcal{I}_{\mathcal{T}, \mathcal{A}} \setminus \mathcal{A}$, the *anonymous nodes*, and we refer to the facts in $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ that mention at least one anonymous node as the *anonymous part* of $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$. It is not hard to see that if there is a single fact $C(a)$ in $\mathcal{A}$, then all the anonymous nodes added during the model construction form a tree-shaped graph rooted at a . Moreover, since all rules are deterministic, this tree is unique up to renaming of nodes, and the canonical model of any ABox containing an individual that satisfies C will contain a copy of this tree.

Our rewriting of a single N2RPQ q is based on two *skipping functions* that take an N2RPE as input and output a modified N2RPE with additional transitions that allow it to ‘skip over’ implicit facts, and instead look for the explicit facts that trigger their generation. The rewriting is done in two phases. First, to account for the anonymous part, the function `skipAnon` adds transitions that skip over the anonymous part of the canonical model. The answers of the resulting query over the facts of $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ involving only non-anonymous individuals coincide with the answers of the original q over the whole $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$. Then a second function `skipCore` adds more transitions to yield complete answers when evaluated over $\mathcal{A}$ alone, without any implicit facts.

3.1 Skipping over the Anonymous Part

Given $\mathcal{T}$ and $\mathcal{A}$, we use $\mathcal{G}_{\mathcal{T}, \mathcal{A}}$ to denote the restriction of the canonical model $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ to the nodes in $\mathcal{A}$, and call it the *core* of $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$. The goal of the first phase of the rewriting is to obtain a query that skips over the anonymous part of $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ and finds all query answers when evaluated over the core $\mathcal{G}_{\mathcal{T}, \mathcal{A}}$ alone.

To this aim, we rely on the following $\text{SkipRel}(\alpha, \mathcal{T})$ relation, originally defined as two relations `JumpTrans` and `JumpFinal` in [8]. This relation characterises the parts of the input N2RPQ α that can be matched in the canonical models of each tree-shaped part that can possibly appear in a canonical model constructed from $\mathcal{T}$.

Recall the definition of run from Definition 7. A *partial run* for α is defined analogously, but does not need to satisfy the conditions on the leafs, i.e. it may simply end on non-final states.

Definition 14. Let $\mathcal{T}$ be a DL-Lite_{PG} TBox and α_0 an n-NFA of depth k , and let $\alpha_1, \dots, \alpha_n$ be the n-NFAs of nesting depth $\leq k$ mentioned in the transitions of α_0 . The set $\text{SkipRel}(\alpha_0, \mathcal{T})$ consists of tuples $(C, s_0^1, s_0^2, \Gamma)$ where $C \in C \cup \{\top\}$, $s_0^1 \in Q_0$ is a state from α_0 , $s_0^2 \in (Q_0 \cup \{f\}) \setminus \{s_0^1\}$ is either a different state from α_0 or the special marker f , and $\Gamma \subseteq \bigcup_{1 \leq j \leq n} Q_j$ is a set of states from the automata nested in α_0 . Such a tuple $(C, s_0^1, s_0^2, \Gamma)$ belongs to $\text{SkipRel}(\alpha_0, \mathcal{T})$ if there exists a partial run (T, l) of α_0 on the canonical model $\mathcal{I}_{\mathcal{T}, \{C(a)\}}$ that satisfies the following conditions:

- The root of T is labelled (a, s_0^1) .

- If $s_0^2 \in Q_0$, there is a leaf node v with $l(v) = (a, s_0^2)$.
- For every leaf node v with $l(v) = (o, s) \neq (a, s_0^2)$, either $s \in F_j$ for some $0 \leq j \leq n$, or $o = a$ and $s \in \Gamma$.

If T has at least one node (o, s) with $o \neq a$, then we call the tuple $(C, s_0^1, s_0^2, \Gamma)$ *strictly anonymous*.

Intuitively, a tuple $(C, s_0^1, s_0^2, \Gamma)$ is in $\text{SkipRel}(\alpha_0, \mathcal{T})$ precisely when in the canonical model of every ABox containing a node a with $a \in C^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$, we are guaranteed to find a match for the query α' obtained by making s_0^1 the initial state of α_0 , s_0^2 the final state, and making all the states in Γ initial states of the respective automata. The match uses only the tree of $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ rooted at a , and it is such that the automata will be at a whenever they visit a state s in Γ (which intuitively acts as initial state in the modified α' but not in the original α_0). This allows us to ‘glue’ together the partial match in the tree rooted at a with any partial matches for the rest of the query found in the rest of the canonical model. Hence, if we find a node that satisfies C , we can fully skip the subquery α' .

The computation of $\text{SkipRel}(\alpha_0, \mathcal{T})$ can be easily reduced to standard description logics reasoning. In the claim below, we use a TBox in the well-known description logic $\mathcal{ELHI}$, where concept inclusions take the form $C \sqsubseteq D$ with C, D concepts formed according to the grammar $C, D := \top \mid A \mid \exists r.C$, and role inclusions take the form $r \sqsubseteq t$, with $A \in \mathbf{C}$ and $r, t \in \bar{\mathbf{R}}$.

LEMMA 1 (PROPOSITION 6.3 IN [8]). Consider an n-NFA α_0 and a DL-Lite_{PG} TBox $\mathcal{T}$, and let $\alpha_1, \dots, \alpha_n$ be the n-NFAs of nesting depth k mentioned in the transitions of α_0 . For every state $s \in Q_i$ of α_i , with $1 \leq i \leq k$, we use a fresh concept name A_s , and we let $A_f = \top$. Let $\mathcal{T}'$ be obtained by keeping only the $\mathcal{ELHI}$ axioms of $\mathcal{T}$, and adding to it the following concept inclusions, for every $\{s, s'\} \subseteq Q_i$, $0 \leq i \leq n$:

- $\top \sqsubseteq A_s$, for every $s \in F_j$ with α_j ,
- $\exists r.A_{s'} \sqsubseteq A_s$ for every $(s, r, s') \in \delta_i$,
- $A_{s'} \sqcap B \sqsubseteq A_s$, whenever $(s, B, s') \in \delta_i$, and
- $A_{s'} \sqcap A_{s''} \sqsubseteq A_s$, whenever $(s, \langle \alpha_j \rangle, s') \in \delta_i$ and $s'' \in S_j$ is an initial state of α_j .

Then the following holds:

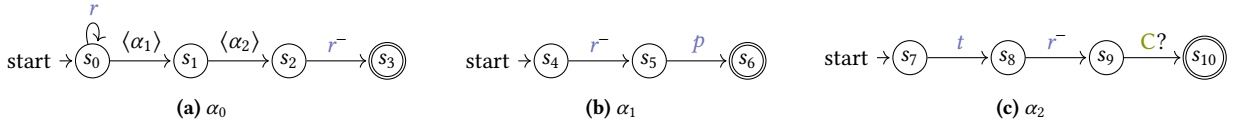
$$(C, s_0^1, s_0^2, \Gamma) \in \text{SkipRel}(\alpha_0, \mathcal{T}) \text{ iff } \mathcal{T}' \models (C \sqcap A_{s_0^2} \sqcap \bigcap_{s \in \Gamma} A_s) \sqsubseteq A_{s_0^1}.$$

Example 15. To illustrate SkipRel , we consider an example n-NFA α_0 , given in Figure 3. We furthermore consider a TBox $\mathcal{T} = \{A \sqsubseteq \exists r.r^-, D \sqsubseteq \exists t'. \top, \exists t'. \top \sqsubseteq C, \{p \geq 10?\} \sqsubseteq D\}$. Then, the following holds:

$$\text{SkipRel}(\alpha_0, \mathcal{T}) = \{(A, s_0, s_3, \{s_5, s_8\}), (A, s_0, s_3, \{s_6, s_9\}), (A, s_5, s_6, \emptyset)\}$$

To see why, for example $(A, s_0, s_3, \{s_5, s_8\})$ is in the set, we note that in the run of α_0 starting from s_0 , we must have a node a in the graph that has an outgoing r edge and by $r^- \sqsubseteq t$ also an incoming t . In state s_5 in α_1 we previously followed a t edge and in state s_8 in α_2 an r^- edge. Thus, the run can just follow that same edge in reverse, landing at a again.

Deciding $\mathcal{T} \models C \sqsubseteq D$ is an instance of a well-studied problem: subsumption in $\mathcal{ELHI}$. It is Exp-complete [26], but efficient off-the-shelf reasoners are available, [6, 25, 38]. However, there are exponentially many possible candidates to populate the set $\text{SkipRel}(\alpha_0, \mathcal{T})$, and testing them all (as assumed in [8]) would require strictly exponentially many calls to the external reasoner,

Figure 3: Example N2RPQ with nesting level $k = 1$.

which is clearly impractical. Fortunately, a simple pruning of the search space can discard many irrelevant candidates. First, we can restrict our attention to tuples that are strictly anonymous, as otherwise the partial runs do not use the anonymous part and are already present in $\mathcal{G}_{\mathcal{T}, \mathcal{A}}$. For the strictly anonymous tuples, we make a simple relevance test: there must be some edge in the canonical model that allows the query to enter the anonymous part and exit it again.

Definition 16. Let $\mathcal{T}$ be a $DL\text{-}Lite_{PG}$ TBox and α_0 an n-NFA of depth k , and let $\alpha_1, \dots, \alpha_n$ be the n-NFAs of nesting depth k mentioned in the transitions of α_0 . A tuple $(C, s_0^1, s_0^2, \Gamma)$ where $C \in C \cup \{\top\}$, $s_0^1, s_0^2 \in Q_0$ and $\Gamma \subseteq \bigcup_j Q_j$, with $1 \leq i, j \leq n$, is called *relevant* if $\{s_0^1\} \cup \Gamma$ does not contain any final states and the tuple satisfies all the following conditions (with ‘ $_$ ’ we denote an arbitrary state).

- (1) $B \sqsubseteq \exists r. \top \in \mathcal{T}$, $r \sqsubseteq_{\mathcal{T}}^* r'$, and $r \sqsubseteq_{\mathcal{T}}^* t$.
- (2) For some $0 \leq i \leq n$, we have $\{(_, r', _) \subseteq \delta_i$.
- (3) If $s_0^2 \neq f$, then for some $0 \leq i \leq n$, we have $\{(_, t^-, _) \subseteq \delta_i$.

LEMMA 2. If $(C, s_0^1, s_0^2, \Gamma) \in \text{SkipRel}(\alpha_0, \mathcal{T})$ and it is strictly anonymous, then it is relevant.

PROOF. Given an n-NFA α_0 , let $\alpha_1, \dots, \alpha_n$ be the n-NFAs of nesting depth k mentioned in the transitions of α_0 . Consider $(C, s_0^1, s_0^2, \Gamma) \in \text{SkipRel}(\alpha_0, \mathcal{T})$ with $\Gamma = \{s_1, \dots, s_n\}$, then we show that each of the conditions in Definition 16 is satisfied. By Definition 14 there is a partial run (T, l) of α_0 in the canonical model $\mathcal{I}$ of $\mathcal{T}$ and $\mathcal{A} = \{C(a)\}$ such that the following are satisfied.

- The root of T is labelled (a, s_0^1) .
- If $s_0^2 \in Q_0$, there is a leaf node v with $l(v) = (a, s_0^2)$.
- For every leaf node v with $l(v) = (o, s) \neq (a, s_0^2)$, either $s \in F_j$ for some $0 \leq j \leq n$, or $o = a$ and $s \in \Gamma$.
- T has at least one node (o, s) with $o \neq a$.

It is not hard to argue that the conditions of Definition 16 hold. Since the construction of canonical model $\mathcal{I}$ is based on the ABox $\mathcal{A} = \{C(a)\}$ with a being the only individual, if there is an individual $o \neq a$, it must have been added by rules (Ch6) or (Ch7) in Definition 13, which requires the existence of an axiom $B \sqsubseteq \exists r. \top \in \mathcal{T}$, as desired, and it is triggered by $a \in B^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$. By construction, $(a, o) \in r^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$. Moreover, for every $r' \in \bar{\mathbf{R}}$, we have that $(a, o) \in r'^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$ implies $r \sqsubseteq_{\mathcal{T}}^* r'$, as (Ch1) is the only rule that can add role names for already existing pairs of nodes. Moreover, the partial run can only visit o after a if it has a transition using such an r' . Hence, conditions (1) and (2) hold. Finally, if $s_0^2 \neq f$ then there is a leaf with label (a, s_0^2) , which means that the partial run returns to a after o , which it can only do using some $t^- \in \bar{\mathbf{R}}$ that occurs in the transition function, and such that $(a, o) \in t^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$, which as argued above, implies $r \sqsubseteq_{\mathcal{T}}^* t$, ensuring that (3) holds. $\square$

Each tuple $(C, s_0^1, s_0^2, \Gamma)$ in $\text{SkipRel}(\alpha_0, \mathcal{T})$ tells us that if our data has a node a that satisfies C , we can skip directly from s_0^1 to s_0^2 ,

since a path that satisfies the transitions between them always exists in the canonical model, specifically in the anonymous part below a . Moreover, any nested automata that started but did not finish their runs in the anonymous part below a will have to ‘exit’ the anonymous part in some state s . These exit states s are stored in Γ , and the corresponding nested automaton only needs to be executed from s onwards to complete its run. We modify our input α_0 to do precisely that: look for a C , jump from s_0^1 to s_0^2 , and still look for partial runs that complete the nested automata, setting the states in Γ as their initial states.

We introduce some notation. For an n-NFA $\alpha = \langle Q, \Sigma_{PG}, S, \delta, F \rangle$ of nesting depth k , we denote by $\alpha_{|s_0}$ the n-NFA $\langle Q, \Sigma_{PG}, \{s_0\}, \delta, F \rangle$, that is, we make s_0 the single initial state.

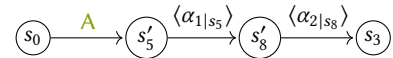
Definition 17 (Skipping Anonymous). Given a $DL\text{-}Lite_{PG}$ TBox $\mathcal{T}$ and an n-NFA $\alpha_0 = \langle Q_0, \Sigma_{PG}, S_0, \delta_0, F_0 \rangle$, let $\alpha_1, \dots, \alpha_n$ be the n-NFAs of nesting depth k mentioned in the transitions of α_0 . Let $(C, s_0^1, s_0^2, \Gamma) \in \text{SkipRel}(\alpha_0, \mathcal{T})$ with $\Gamma = \{s_1, \dots, s_n\}$. We associate a state to s_0^2 as follows: if $s_0^2 \in Q_0$, then $\text{state}(s_0^2) = s_0^2$, and if $s_0^2 = f$, then $\text{state}(s_0^2)$ is a fresh state s_f . If $\Gamma = \emptyset$, we use s'_1 to denote $\text{state}(s_0^2)$, and for each $s_j \in \Gamma$, we let α^j denote the α_i with $s_j \in Q_j$. Then $\text{skipAnon}^{\mathcal{T}}(\alpha_0) = \langle Q'_0, \Sigma_{PG}, S_0, \delta'_0, F'_0 \rangle$ is the n-NFA with:

$$\begin{aligned} Q'_0 &= Q_0 \cup \{\text{state}(s_0^2)\} \cup \{s'_j \mid s_j \in \Gamma\} \\ F'_0 &= F_0 \cup \{s_f\} \text{ if } s_0^2 = f \text{ and } F_0 = F_0 \text{ otherwise, and} \\ \delta'_0 &= \delta_0 \cup \{(s_0^1, C?, s'_1)\} \cup \\ &\quad \{(s'_1, \langle \alpha_{|s_1}^1 \rangle, s'_2)\} \cup \dots \cup \{(s'_{n-1}, \langle \alpha_{|s_{n-1}}^{n-1} \rangle, s'_n)\} \cup \\ &\quad \{(s'_n, \langle \alpha_{|s_n}^n \rangle, s_0^2)\} \end{aligned}$$

In particular if $\Gamma = \emptyset$, the only new transition is $\{(s_0^1, C?, \text{state}(s_0^2))\}$.

We can show that $\text{skipAnon}^{\mathcal{T}}(\alpha)$ correctly skips over the anonymous part of $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ and finds all query answers when evaluated over the core $\mathcal{G}_{\mathcal{T}, \mathcal{A}}$ alone.

Example 18. Consider the N2RPQ from Figure 3 and the TBox $\mathcal{T}$ from Example 15. Then, by $\text{skipAnon}^{\mathcal{T}}(\alpha_0)$ and for the tuple $(A, s_0, s_3, \{s_5, s_8\}) \in \text{SkipRel}(\alpha_0, \mathcal{T})$ we add the following transitions and fresh states to α_0 .



LEMMA 3. Let $\mathcal{T}$ be a $DL\text{-}Lite_{PG}$ TBox and α be an N2RPE. Then, for every PG $\mathcal{A}$ and every pair of nodes o_s, o_f from $\mathcal{A}$, it holds that $(o_s, o_f) \in \alpha^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$ iff $(o_s, o_f) \in \text{skipAnon}^{\mathcal{T}}(\alpha)^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$.

PROOF SKETCH. The proof is contained in the proof of Proposition 6.6 in [8]; there is a one-to-one correspondence between the nested automata added by $\text{skipAnon}^{\mathcal{T}}(\mathcal{T})(\alpha_0)$ and the recursive calls to EvalAtom in cases (3.b.ii) and (3.b.iii). Only a minor adaptation is needed to argue that soundness and completeness are not

compromised if the tests are restricted to strictly anonymous, relevant tuples in SkipRel. The rest of the evaluation, which is done only over the individuals in $\mathcal{A}$, coincides exactly with the standard evaluation over $\mathcal{G}_{\mathcal{T}, \mathcal{A}}$. $\square$

3.2 Rewriting over the Core

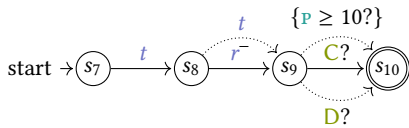
We know that $\text{skipAnon}^{\mathcal{T}}(\alpha)$ finds all query answers when evaluated over $\mathcal{G}_{\mathcal{T}, \mathcal{A}}$, but we want to execute the rewritten query over $\mathcal{A}$ alone, that is, over the explicit facts only. Again, we achieve this by adding transitions that allow us to look for explicit facts only and skip over their consequences. As usual for DL-Lite variants, in the canonical model obtained from a $\text{DL-Lite}_{\text{PG}}$ TBox, every implicit fact ξ_ℓ is implied by some other fact $\xi_{\ell-1}$, which is either explicit or is also implied by another fact $\xi_{\ell-2}$ and so on, until we reach some explicit fact ξ_0 that triggered the existence of ξ_ℓ . We add transitions to α_0 that allow it to use any of these ξ_i when looking for some ξ_ℓ .

Definition 19 (Skipping Core). Let $\mathcal{T}$ be a $\text{DL-Lite}_{\text{PG}}$ TBox, and let $\mathcal{T}^*$ be the result of closing $\mathcal{T}$ under the following rule:

- ($\star$) If $\{C \sqsubseteq \exists r. \top, \exists r'. \top \sqsubseteq D\} \subseteq \mathcal{T}^*$ and $r \sqsubseteq_{\mathcal{T}} r'$, then $C \sqsubseteq D \in \mathcal{T}^*$. Given a N2RPE α with alphabet Σ_{PG} , we denote by $\text{skipCore}^{\mathcal{T}}(\alpha)$ the n-NFA obtained by exhaustively adding transitions as follows. In each rule, δ, s_i, s_j and s_k denote the transition function and states of one (arbitrary but fixed) automaton that occurs (possibly nested) in α , and as usual, $r, p \in \bar{\mathbf{R}}, C, D \in \mathbf{C}$ and $T = \{t_1, \dots, t_k\} \subseteq \mathbf{T}^{\mathbf{D}}$.
- (R1) If $r \sqsubseteq p \in \mathcal{T}$ and $(s_i, S, s_j) \in \delta$ with $p \in S$ (or $p^- \in S$), then add (s_i, R, s_j) to δ with $R = (S \setminus \{p\}) \cup \{r\}$ (or $R = (S \setminus \{p^-\}) \cup \{r^-\}$).
 - (R2) If $T \sqsubseteq r \in \mathcal{T}$ and $(s_i, R, s_j) \in \delta$ with $r \in R$, add (s_i, T, s_j) to δ .
 - (R3) If $C \sqsubseteq D \in \mathcal{T}^*$ and $(s_i, D?, s_j) \in \delta$, then add $(s_i, C?, s_j)$ to δ .
 - (R4) If $\exists r. \top \sqsubseteq B \in \mathcal{T}^*$ and $(s_i, B?, s_j) \in \delta$, then add $(s_i, \langle \alpha_r \rangle, s_j)$ to δ for a fresh two-state NFA α_r with a single transition $\delta_r(s_0^1, \{r\}, s_f)$ between its initial state s_0^1 and its final state s_f .
 - (R5) If $T \sqsubseteq D \in \mathcal{T}^*$ and $(s_i, D?, s_j) \in \delta$, then add, for each $t_\ell \in T$, a transition $(s'_\ell, t_\ell?, s'_{\ell+1})$ to δ , where $s_i = s'_1$, $s'_{k+1} = s_j$, and $s'_2, \dots, s'_k$ are fresh states.

Intuitively, each rule application adds a transition that allows α to ‘skip’ an implicit fact ξ_j in $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ and use instead a fact ξ_i , $i < j$, that participated in its creation. We illustrate $\text{skipCore}^{\mathcal{T}}(\alpha)$ on a simple example.

Example 20. Let us again consider the N2RPE α_2 from Figure 3 and the TBox $\mathcal{T}$ from Example 15. Then, by $\text{skipCore}^{\mathcal{T}}(\alpha_2)$ we add the dotted transitions shown below.



We can now show that $\text{skipCore}^{\mathcal{T}}(\alpha)(x, y)$, when evaluated over $\mathcal{A}$, correctly accounts for the implicit facts in $\mathcal{G}_{\mathcal{T}, \mathcal{A}}$.

LEMMA 4. Let $\mathcal{T}$ be a $\text{DL-Lite}_{\text{PG}}$ TBox and α be an N2RPE. Then, for every PG $\mathcal{A}$ and every pair of nodes o_s, o_f from $\mathcal{A}$, it holds that $(o_s, o_f) \in \alpha^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$ iff $(o_s, o_f) \in \text{skipCore}^{\mathcal{T}}(\alpha)^{\mathcal{A}}$.

PROOF SKETCH. Let $\mathcal{T}^*_{\text{core}}$ be the restriction of $\mathcal{T}^*$ to the axioms that do not have existential concepts on the right, and for readability, for a given $\mathcal{A}$, we denote by $\mathcal{A}^*$ the canonical model $\mathcal{I}_{\mathcal{T}^*_{\text{core}}, \mathcal{A}}$.

One can prove in the usual way (see, e.g., [19]) that $\mathcal{G}_{\mathcal{T}, \mathcal{A}} = \mathcal{A}^*$. We can thus show that, for every α :

$$\alpha^{\mathcal{A}^*} = (\text{skipCore}^{\mathcal{T}}(\alpha))^{\mathcal{A}}.$$

Given the one-to-one correspondence between the rules (Ch1) to (Ch5), which add facts to $\mathcal{A}^*$ and the rules (R1) to (R5), which add transitions to $\text{skipCore}^{\mathcal{T}}(\alpha)$, the equality is established by a simple rule-by-rule analysis. $\square$

Together, Lemma 3 and Lemma 4 give us the desired rewriting.

PROPOSITION 1. Let $\mathcal{T}$ be a $\text{DL-Lite}_{\text{PG}}$ TBox and α be an N2RPE. Then, for every PG $\mathcal{A}$ and every pair of nodes o_s, o_f from $\mathcal{A}$, it holds that $(o_s, o_f) \in \alpha^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$ iff $(o_s, o_f) \in \text{skipCore}^{\mathcal{T}}(\text{skipAnon}^{\mathcal{T}}(\alpha))^{\mathcal{A}}$.

4 Rewriting join-on-free CN2RPFs

Utilising the two skip procedures from Section 3, we can now present an algorithm for the rewriting of join-on-free CN2RPFs under $\text{DL-Lite}_{\text{PG}}$ ontologies. We dub this procedure ‘Automata-based Rewriting of CN2RPFs under Ontologies’, or ARCO for short. We proceed to describe the key parts of the procedure.

We use a function that inverts nested automata, that is, it transforms the automata to accept the same words, but inverted.

Definition 21. We define an *inverse* function over the alphabet $\Sigma_{\text{PG}} \cup \bigcup_{k \in \mathbb{N}} \mathbf{A}^k$: if $\sigma \notin 2^{\bar{\mathbf{R}} \cup \mathbf{T}^{\mathbf{D}}}$, then $\sigma^- = \sigma$, and if $\sigma \in 2^{\bar{\mathbf{R}} \cup \mathbf{T}^{\mathbf{D}}}$, then $\sigma^- = \{t \mid t \in \sigma \cap \mathbf{T}^{\mathbf{D}}\} \cup \{r^- \mid r \in \sigma \cap \bar{\mathbf{R}}\} \cup \{r \mid r^- \in \sigma, r \in \bar{\mathbf{R}}\}$.

Let $\alpha = \langle Q, \Sigma_{\text{PG}}, S, \delta, F \rangle$ be an N2RPE with $S \subseteq Q$ the initial states and $F \subseteq Q$ are the final states. Then α can be *inverted* to obtain $\bar{\alpha} = \langle Q, \Sigma_{\text{PG}}, F, \delta^-, S \rangle$, where $\delta^- = \{(s_j, \sigma^-, s_i) \mid (s_i, \sigma, s_j) \in \delta\}$.

LEMMA 5. For every N2RPE α , PG $\mathcal{I}$, and pairs of nodes n_1, n_2 , $(n_1, n_2) \in \alpha^{\mathcal{I}}$ if and only if $(n_1, n_2) \in (\bar{\alpha})^{\mathcal{I}}$.

The first step of our rewriting algorithm removes all non-answer variables from the query by making use of nested expressions.

Definition 22. For a join-on-free CN2RPF $q(\vec{x})$, we denote by $q^{(\exists)}(\vec{x})$ the result of replacing each atom $\alpha(x, y)$ with $y \notin \vec{x}$ by $\langle \alpha \rangle(x)$, and each $\alpha(x, y)$ with $x \notin \vec{x}$ by $\langle \bar{\alpha} \rangle(y)$.

A simple inspection of the CN2RPF semantics shows that the transformation preserves query answers.

LEMMA 6. Let $q(\vec{x})$ be a join-on-free CN2RPF. For every interpretation $\mathcal{I}$ and tuple of nodes $\vec{a}$, we have that $\vec{a}$ is an answer to $q(\vec{x})$ in $\mathcal{I}$ iff $\vec{a}$ is an answer to $q^{(\exists)}(\vec{x})$ in $\mathcal{I}$.

We emphasise again that the transformation to $q^{(\exists)}$ only removes non-answer variables and leaves all other variables untouched. We denote by $\text{arco}^{\mathcal{T}}(q^{(\exists)}(\vec{x}))$ the result of executing Algorithm 1 on $q(\vec{x})$ and $\mathcal{T}$. As we can see, it begins with the removal of non-answer variables using $q^{(\exists)}$, and then applies $\text{skipCore}^{\mathcal{T}}(\alpha)$ and $\text{skipAnon}^{\mathcal{T}}(\alpha)$ to each atom in $q^{(\exists)}(\vec{x})$.

Next we show that Algorithm 1 is indeed sound and complete.

THEOREM 23. Let $\mathcal{T}$ be a $\text{DL-Lite}_{\text{PG}}$ TBox and $q(\vec{x})$ be a join-on-free CN2RPF. Then, for every ABox $\mathcal{A}$ and every tuple $\vec{a}$ of individuals from $\mathcal{A}$, it holds that $\vec{a}$ is a certain answer to $q(\vec{x})$ over $(\mathcal{T}, \mathcal{A})$ iff $\vec{a}$ is an answer to $\text{arco}^{\mathcal{T}}(q^{(\exists)}(\vec{x}))$ over $\mathcal{A}$.

Algorithm 1: Automata-based Rewriting of CN2RPQs under Ontologies (ARCO)

Input : join-on-free CN2RPQ $q(\vec{x})$, TBox $\mathcal{T}$
Output: CN2RPQ q'

```

1 function arco( $q$ ):
2    $q = \text{remove}\exists\text{Var}(q)$ 
3   while  $q \neq q'$  do
4      $q' = q$ 
5     foreach  $\alpha(x, y) \in q$  do
6        $q' = q'[\alpha \backslash \text{skipAnon}^{\mathcal{T}}(\alpha)]$ 
7        $q' = q'[\alpha \backslash \text{skipCore}^{\mathcal{T}}(\alpha)]$ 
8   return  $q$ 
9 function remove $\exists\text{Var}(q)$ :
10  foreach  $\alpha(x, y) \in q$  do
11    if  $y \notin \vec{x}$  then
12       $q = q[\alpha(x, y) \setminus \langle \alpha \rangle(x)]$ 
13    else if  $x \notin \vec{x}$  then
14       $q = q[\alpha(x, y) \setminus \langle \alpha^- \rangle(y)]$ 
15  return  $q$ 

```

PROOF. Given a $DL\text{-}Lite_{PG}$ TBox $\mathcal{T}$ and an ABox $\mathcal{A}$ and a join-on-free CN2RPQ $q(\vec{x})$. From For ($\Rightarrow$) assume that $\vec{a}$ of individuals from $\mathcal{A}$ is a certain answer to $q(\vec{x})$ over $(\mathcal{T}, \mathcal{A})$. From Lemma 6 it holds that $\vec{a}$ is a certain answer to $q^{(\exists)}(\vec{x})$ over $(\mathcal{T}, \mathcal{A})$, i.e., $\vec{a} = \mu(\vec{x})$ and $(\mu(x), \mu(y)) \in \alpha^{I_{\mathcal{T}, \mathcal{A}}}$ (or $(\mu(x) \in \alpha^{I_{\mathcal{T}, \mathcal{A}}})$ for each atom $\alpha(x, y)$ (or $\alpha(x)$) in $q^{(\exists)}(\vec{x})$). By Lemma 4 we can imply that $(\mu(x), \mu(y)) \in \text{arco}^{\mathcal{T}}(\alpha)^{\mathcal{A}}$ and $(\mu(x) \in \text{arco}^{\mathcal{T}}(\alpha)^{\mathcal{A}})$ respectively. Thus, the claim holds that $\vec{a}$ is an answer to $\text{arco}^{\mathcal{T}}(q^{(\exists)}(\vec{x}))$ over $\mathcal{A}$. Since the claims in Lemma 4 and Lemma 6 hold in both directions, the proof for ($\Leftarrow$) is analogous. $\square$

Example 24. For the algorithm ARCO the only new step in this section is the removal of existential variables. For demonstration purposes consider the query $q(x, y) := \alpha_1(x, y), \alpha_2(x_1, y), \alpha_3(x, y_1)$. Then, the returned query of $\text{remove}\exists\text{Var}(q)$ is $q(x, y) := \alpha_1(x, y), \langle \alpha_2^- \rangle(y), \langle \alpha_3 \rangle(x)$. Since for $\alpha_2(x_1, y)$ variable y on the right side is bound, we have to invert α_2 . If we consider α_2 to be from Figure 3c, then the following is the inverse α_2^- .



5 Implementation & Experiments

In this section, our aim is to show the practical utility of our rewriting algorithm ARCO (cf. Section 4). To this end, we implemented a prototype of our rewriting algorithm in Java. This prototype takes as input a navigational query (join-on-free CN2RPQ), using a custom format, and rewrites it with respect to a $DL\text{-}Lite_{PG}$ ontology (our DL-Lite extension supporting tests over property values) expressed in the OWL format. As part of the experiments, we then run the produced Cypher queries over a Neo4j database and report on the results. We share the source code of our prototype anonymously⁵ and will make it publicly available in the future.

⁵<https://anonymous.4open.science/r/owl2cypher/README.md>

Table 2: Statistics of ontologies and Neo4j data

ontology	# concepts	# roles	Neo4j data	
			#nodes	#relations
DBPedia	1230	1212	10 075 182	22 194 296
MMM	53	18	107 624	379 002

Setup and Implementation. All experiments were executed on a virtual machine with 400 GB RAM and 16 cores running Debian GNU/Linux 12 (Bookworm). The host machine is equipped with an AMD EPYC 7313 processor clocked at 2.00 GHz. Neo4j 5.25.0 was deployed on the same virtual machine.

We implemented a prototype of our rewriting algorithm ARCO in Java. We base our implementation on several external libraries: the OWL API [24] to parse and extend OWL ontologies, the Hermit reasoner [23] to verify the JumpTrans candidates, ANTLR [34] for parsing the input CN2RPQs, the Java library JgraphT [30] for the internal representation of n-NFAs, and finally Cypher DSL [36] to convert the output into Cypher syntax.

Ontology and Data. To evaluate our rewriting algorithm, we considered two real-world ontologies from different domains with divergent characteristics. In Table 2 we provide the number of concepts and roles for each ontology. DBPedia [29] is a dataset containing extracted data from Wikipedia articles, which is of interest to our rewriting algorithm due to its size. The second dataset is the open data of Montpellier Méditerranée Métropole [7] (MMM), which covers the sewer system of the city. This ontology contains existentials on the right-hand side, which triggers $\text{skipAnon}^{\mathcal{T}}(\alpha_0)$ from Section 3.

To the best of our knowledge, the current version of the OWL2 standard [39] does not support data tests in role inclusions, i.e., axioms of the form $\{p \odot v\} \sqsubseteq r$; therefore, we do not consider this kind of axioms in the evaluation.

Each ontology in Table 2 is associated with a corresponding dataset stored in a separate Neo4j database. We choose Neo4j because it provides a level of GQL compliance sufficient for our experimental evaluation. We relate concepts in the ontology to node labels in the database, while roles are related to relationship types, i.e., edge labels.

Queries. Since no benchmark queries are available for rewriting join-on-free CN2RPQs, we adopt two approaches for constructing evaluation queries: (1) to evaluate the rewriting algorithm we synthetically generate 100 N2RPQs by randomly selecting concepts and roles from a given ontology, and by randomly applying operators such as concatenation, union, and the Kleene star, while enforcing structural constraints such as bounded path lengths and nesting levels; (2) for the evaluation with Neo4j of the rewritten queries we additionally handcraft a set of 5 queries based on the MMM dataset and the SPARQL queries therein.

Results. Figure 4 shows the results of rewriting the generated queries. Notice in these figures we report the results as pairs: for each query, we show its transitions (seen as an N2RPE) before and after rewriting, on the x axis. Both data points share the same y-axis,

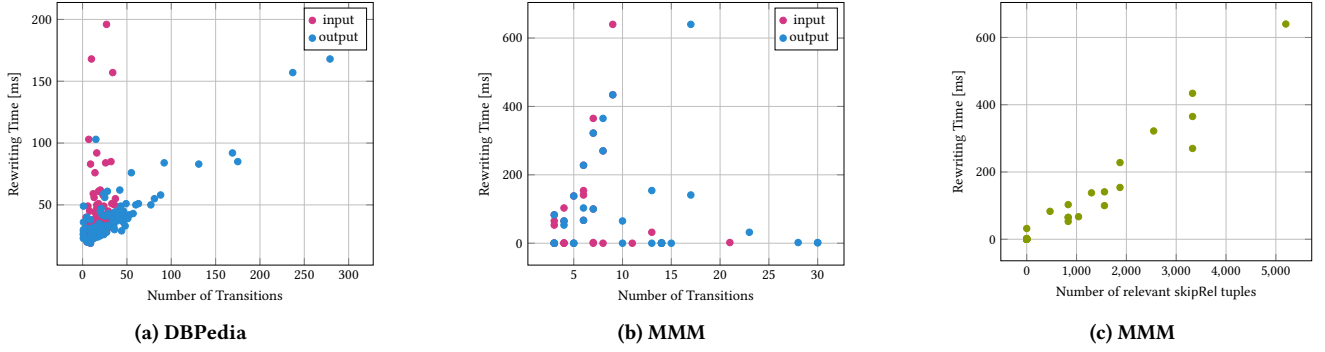


Figure 4: Runtime of the rewriting algorithm.

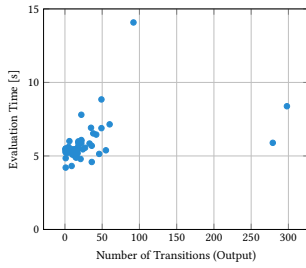


Figure 5: Evaluation of DBPedia dataset. 56 timeouts at 20s.

indicating the rewriting time. Although the DBPedia ontology contains significantly more concepts and roles, rewriting is generally faster (under 250 ms) than with the MMM ontology (around 600 ms). This is because DBPedia does not contain any axioms with existentials on the right-hand side ($A \sqsubseteq \exists r.T$). Therefore, there are no relevant SkipRel tuples. As can be seen from Figure 4a, the rewriting time depends on the number of transitions in the rewritten query for DBPedia. From the results with the MMM dataset, we can conclude that, if there are relevant SkipRel tuples, they seem to impact the rewriting time more than a high number of concepts (see Figure 4b and Figure 4c). It should be noted that for 62 of the 100 generated queries computing the power set was not feasible given the number of states of the query (≥ 30).

While the evaluation of rewriting times already shows practical feasibility, the generated queries with Neo4j still seems to be a challenge, as suggested by the results in Table 3 and Figure 5.

In the DBPedia dataset, we see queries falling into two groups: either running within 2 to 8 seconds or quickly timing out (with the timeout at 20s). We see only a weak correlation with number of transitions in the output. Similarly, 3 of 5 queries over the MMM dataset time out.

The reasons for the high evaluation times may be, on the one hand, query constructions that initiate the computation of the Cartesian product or, on the other hand, a high number of query answers. Note, that the generated queries do not return any answers, as could be anticipated. However, optimising the evaluation of the generated queries was out of scope. From these results, it is clear that there is a need to understand exactly why rewritten queries time-out and how to achieve optimal rewritten queries.

Table 3: Evaluation of MMM dataset. Timeout($\downarrow$) at 1000s.

query	transitions		# skipRel	time [ms]	
	input	output		rew.	eval.
Q1	6	6	52	153	52 114
Q2	6	9	468	137	$\downarrow$
Q3	3	6	52	42	377 845
Q4	5	20	624	69	$\downarrow$
Q5	4	10	312	41	$\downarrow$

6 Conclusion

In this paper, we present the first practical algorithm for rewriting N2RPQs and a significant subset of CN2RPQs, making use of the ability of modern graph databases to express complex navigational queries with nesting. We particularly note the ability of our approach to access key values within path expressions in the presence of an ontology with data tests. The result is a highly flexible approach to ontology-mediated querying of property graphs that still allows for query rewriting into native graph database technologies. We have demonstrated the formal correctness of our approach and provided a proof-of-concept implementation that can rewrite Cypher queries to incorporate ontological knowledge and evaluate the rewritten queries using Cypher.

To achieve a simple and practicable solution we focused on join-on-free CN2RPQs. This does not seem too limiting: even plain N2RPEs can already express many realistic examples that involve complex bidirectional navigation on the anonymous implicit facts; arbitrary conjunctions over the free variables make the query language even more powerful. We expect that real-world queries that require different regular paths to travel separately and join somewhere in the unnamed part of the canonical model will very rarely emerge in practice, if at all. We stress that algorithms for full extended CN2RPQs in expressive DLs have been available for over a decade [8]. It is not difficult to adapt these algorithms and extend our technique to be complete for all CN2RPQs and *DL-Lite_{PG}* ontologies; in fact, we do that in the extended version of this paper. However, this seems to dramatically increase the run time of the rewriting algorithm, and the added value seems limited, as we have not yet found a good use case for queries beyond our join-on-free fragment. We explore this further.

Other promising extensions that we plan to explore include improved the support for property values and more expressive ontology languages. For the former, we will consider existing works on query rewriting with data values [5, 35]. For the latter, we will build on recent efforts to identify richer ontology languages with NL data complexity [3, 17, 28].

This work brings us significantly closer to making ontology-based data-access over graph databases a reality, and as such, we aim to explore the potential of our technique for real-life OBDA systems with navigational capabilities, and to conduct experiments on systems supporting GQL as soon as they become available. This includes the work of extending our data model to align more closely with PGs and also to explore practical methods to go beyond the join-on-free fragment.

Another interesting line of future work would be the creation of useful benchmarks that combine both complex navigational queries with rich ontologies expressed over them.

Acknowledgments

This research was funded in whole or in part by the Austrian Science Fund (FWF) 10.55776/COE12, 10.55776/PIN8884924 and 10.55776/P34962. For open access purposes, the author has applied a CC BY public copyright license to any author accepted manuscript version arising from this submission. This work was partially supported by the State of Salzburg under grant number 20102-F2101143-FPR (DNI). We thank the reviewers for their insightful comments and Jean Christoph Jung for pointing out a crucial issue with the completeness of the rewriting algorithm in a preliminary version of this paper.

References

- [1] Renzo Angles. 2018. The Property Graph Database Model. In *Proceedings of the 12th Alberto Mendelzon International Workshop on Foundations of Data Management, Cali, Colombia, May 21-25, 2018 (CEUR Workshop Proceedings, Vol. 2100)*, Dan Olteanu and Barbara Poblete (Eds.). CEUR-WS.org. <https://ceur-ws.org/Vol-2100/paper26.pdf>
- [2] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan L. Reutter, and Domagoj Vrgoc. 2017. Foundations of Modern Query Languages for Graph Databases. *ACM Comput. Surv.* 50, 5 (2017), 68:1–68:40. doi:10.1145/3104031
- [3] János Arpási, Bartosz Bednarczyk, and Magdalena Ortiz. 2025. A Horn Extension of DL-Lite with NL Data Complexity. In *Proceedings of the International Workshop on Description Logics (DL 2025)*. To appear.
- [4] Alessandro Artale, Vladislav Ryzhikov, and Roman Kontchakov. 2012. DL-Lite with Attributes and Datatypes. In *ECAI 2012 - 20th European Conference on Artificial Intelligence. Including Prestigious Applications of Artificial Intelligence (PAIS-2012) System Demonstrations Track, Montpellier, France, August 27-31, 2012 (Frontiers in Artificial Intelligence and Applications, Vol. 242)*, Luc De Raedt, Christian Bessière, Didier Dubois, Patrick Doherty, Paolo Frasconi, Fredrik Heintz, and Peter J. F. Lucas (Eds.). IOS Press, 61–66. doi:10.3233/978-1-61499-098-7-61
- [5] Franz Baader, Stefan Borgwardt, and Marcel Lippmann. 2017. Query Rewriting for DL-Lite with n-ary Concrete Domains. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, Carles Sierra (Ed.). ijcai.org, 786–792. doi:10.24963/IJCAI.2017/109
- [6] Andrew Bate, Boris Motik, Bernardo Cuenca Grau, Frantisek Simancik, and Ian Horrocks. 2016. Extending Consequence-Based Reasoning to SRIQ. In *Principles of Knowledge Representation and Reasoning: Proceedings of the Fifteenth International Conference, KR 2016, Cape Town, South Africa, April 25-29, 2016*, Chitta Baral, James P. Delgrande, and Frank Wolter (Eds.). AAAI Press, 187–196. <https://aaai.org/papers/20-12882-extending-consequence-based-reasoning-to-sriq/>
- [7] Haydar Batoul, Nanée Chahinian, and Claude Pasquier. 2024. From Standards to an Ontology-Based Data Access system for Sewer Networks. In *10ème Journées Doctorales en Hydrologie Urbaine*.
- [8] Meghyn Bienvenu, Diego Calvanese, Magdalena Ortiz, and Mantas Simkus. 2014. Nested Regular Path Queries in Description Logics. In *Principles of Knowledge Representation and Reasoning: Proceedings of the Fourteenth International Conference, KR 2014, Vienna, Austria, July 20-24, 2014*, Chitta Baral, Giuseppe De Giacomo, and Thomas Eiter (Eds.). AAAI Press. <http://www.aaai.org/ocs/index.php/KR/KR14/paper/view/8000>
- [9] Meghyn Bienvenu and Magdalena Ortiz. 2015. Ontology-Mediated Query Answering with Data-Tractable Description Logics. In *Reasoning Web. Web Logic Rules - 11th International Summer School 2015, Berlin, Germany, July 31 - August 4, 2015, Tutorial Lectures (Lecture Notes in Computer Science, Vol. 9203)*, Wolfgang Faber and Adrian Paschke (Eds.). Springer, 218–307. https://doi.org/10.1007/978-3-319-21768-0_9
- [10] Meghyn Bienvenu, Magdalena Ortiz, and Mantas Simkus. 2015. Regular Path Queries in Lightweight Description Logics: Complexity and Algorithms. *J. Artif. Intell. Res.* 53 (2015), 315–374. doi:10.1613/jair.4577
- [11] Angela Bonifati, George H. L. Fletcher, Hannes Voigt, and Nikolay Yakovets. 2018. *Querying Graphs*. Morgan & Claypool Publishers. doi:10.2200/S00873ED1V01Y201808DTM051
- [12] Diego Calvanese, Thomas Eiter, and Magdalena Ortiz. 2009. Regular Path Queries in Expressive Description Logics with Nominals. In *IJCAI 2009, Proceedings of the 21st International Joint Conference on Artificial Intelligence, Pasadena, California, USA, July 11-17, 2009*, Craig Boutilier (Ed.). 714–720. <http://ijcai.org/Proceedings/09/Papers/124.pdf>
- [13] Diego Calvanese, Giuseppe De Giacomo, Domenico Lembo, Maurizio Lenzerini, and Riccardo Rosati. 2007. Tractable Reasoning and Efficient Query Answering in Description Logics: The DL-Lite Family. *Journal of Automated Reasoning* 39, 3 (20 jul 2007), 385–429. doi:10.1007/s10817-007-9078-x
- [14] Diego Calvanese, Giuseppe De Giacomo, Maurizio Lenzerini, and Moshe Y. Vardi. 1999. Rewriting of Regular Expressions and Regular Path Queries. In *Proceedings of the Eighteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, May 31 - June 2, 1999, Philadelphia, Pennsylvania, USA*, Victor Vianu and Christos H. Papadimitriou (Eds.). ACM Press, 194–204. doi:10.1145/303976.303996
- [15] Alin Deutsch, Nadime Francis, Alastair Green, Keith Hare, Bei Li, Leonid Libkin, Tobias Lindaaeker, Victor Marsault, Wim Martens, Jan Michels, Filip Murlak, Stefan Plantikow, Petra Selmer, Oskar van Rest, Hannes Voigt, Domagoj Vrgoc, Mingxi Wu, and Fred Zemke. 2022. Graph Pattern Matching in GQL and SQL/PGQ. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 2246–2258. doi:10.1145/3514221.3526057
- [16] Mirko Michele Dimartino, Andrea Cali, Alexandra Poulouvasilis, and Peter T. Wood. 2016. Query Rewriting under Linear EL Knowledge Bases. In *Web Reasoning and Rule Systems - 10th International Conference, RR 2016, Aberdeen, UK, September 9-11, 2016, Proceedings (Lecture Notes in Computer Science, Vol. 9898)*, Magdalena Ortiz and Stefan Schlobach (Eds.). Springer, 61–76. https://doi.org/10.1007/978-3-319-45276-0_6
- [17] Mirko Michele Dimartino, Peter T. Wood, Andrea Cali, and Alexandra Poulouvasilis. 2025. Efficient Ontology-Mediated Query Answering: Extending DL-liteR and Linear ELH. *J. Artif. Intell. Res.* 82 (2025), 851–899. doi:10.1613/JAIR.1.16401
- [18] Nikola Dragovic, Cem Okulmus, and Magdalena Ortiz. 2023. Rewriting Ontology-Mediated Navigational Queries into Cypher. In *Proceedings of the 36th International Workshop on Description Logics (DL 2023) co-located with the 20th International Conference on Principles of Knowledge Representation and Reasoning and the 21st International Workshop on Non-Monotonic Reasoning (KR 2023 and NMR 2023), Rhodes, Greece, September 2-4, 2023 (CEUR Workshop Proceedings, Vol. 3515)*, Oliver Kutz, Carsten Lutz, and Ana Ozaki (Eds.). CEUR-WS.org. <https://ceur-ws.org/Vol-3515/paper-9.pdf>
- [19] Thomas Eiter, Magdalena Ortiz, Mantas Simkus, Trung-Kien Tran, and Guohui Xiao. 2012. Query Rewriting for Horn-SHIQ Plus Rules. In *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence, July 22-26, 2012, Toronto, Ontario, Canada*, Jörg Hoffmann and Bart Selman (Eds.). AAAI Press, 726–733. <http://www.aaai.org/ocs/index.php/AAAI/AAAI12/paper/view/4931>
- [20] Nadime Francis, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova, and Domagoj Vrgoc. 2022. GPC: A Pattern Calculus for Property Graphs. *CoRR* abs/2210.16580 (2022). arXiv:2210.16580 doi:10.48550/arXiv.2210.16580
- [21] Nadime Francis, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova, and Domagoj Vrgoc. 2023. A Researcher's Digest of GQL (Invited Talk). In *26th International Conference on Database Theory, ICDT 2023, March 28-31, 2023, Ioannina, Greece (LIPIcs, Vol. 255)*, Floris Geerts and Brecht Van de Voort (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 1:1–1:22. doi:10.4230/LIPIcs.ICDT.2023.1
- [22] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaeker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 1433–1445. doi:10.1145/3183713.3190657

- [23] Birte Glimm, Ian Horrocks, Boris Motik, Giorgos Stoilos, and Zhe Wang. 2014. HermiT: An OWL 2 Reasoner. *J. Autom. Reason.* 53, 3 (2014), 245–269. doi:10.1007/S10817-014-9305-1
- [24] Matthew Horridge, Sean Bechhofer, and Olaf Noppens. 2007. The OWL API. <https://owlcs.github.io/owlapi/>. Accessed: 2026-05-21.
- [25] Yevgeny Kazakov. 2009. Consequence-Driven Reasoning for Horn SHIQ Ontologies. In *IJCAI 2009, Proceedings of the 21st International Joint Conference on Artificial Intelligence, Pasadena, California, USA, July 11-17, 2009*, Craig Boutilier (Ed.). 2040–2045. <http://ijcai.org/Proceedings/09/Papers/336.pdf>
- [26] Markus Krötzsch, Sebastian Rudolph, and Pascal Hitzler. 2013. Complexities of Horn Description Logics. *ACM Trans. Comput. Log.* 14, 1 (2013), 2:1–2:36. doi:10.1145/2422085.2422087
- [27] Bianca Löhnert, Nikolaus Augsten, Cem Okulmus, and Magdalena Ortiz. 2025. Query Rewriting for Nested Navigational Queries over Property Graphs. In *Proceedings of the 38th International Workshop on Description Logics - DL 2025, Opole, Poland, September 3-6, 2025 (CEUR Workshop Proceedings)*, Lidia Tendera, Yazmín Ibáñez-García, and Patrick Koopmann (Eds.). CEUR-WS.org. <https://ceur-ws.org/Vol-4091/paper40.pdf>
- [28] Bianca Löhnert, Nikolaus Augsten, Cem Okulmus, and Magdalena Ortiz. 2025. Towards Practicable Algorithms for Rewriting Graph Queries beyond DL-Lite. In *The Semantic Web - 22nd International Conference, ESWC 2025, Portorož, Slovenia, June 1-5, 2025 (accepted for publication) (Lecture Notes in Computer Science)*. Springer.
- [29] Pablo N Mendes, Max Jakob, and Christian Bizer. 2012. *DBpedia: A multilingual cross-domain knowledge base*. European Language Resources Association (ELRA).
- [30] Dimitrios Michail, Joris Kinable, Barak Naveh, and John V. Sichi. 2020. JGraphT – A Java Library for Graph Data Structures and Algorithms. <https://jgrapht.org/>. Accessed: 2026-05-21.
- [31] Neo4j Contributors. [n.d.]. GQL Conformance - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/appendix/gql-conformance/>. Accessed: 2026-05-23.
- [32] Magdalena Ortiz, Sebastian Rudolph, and Mantas Simkus. 2011. Query Answering in the Horn Fragments of the Description Logics SHOIQ and SROIQ. In *IJCAI 2011, Proceedings of the 22nd International Joint Conference on Artificial Intelligence, Barcelona, Catalonia, Spain, July 16-22, 2011*, Toby Walsh (Ed.). IJCAI/AAAI, 1039–1044. doi:10.5591/978-1-57735-516-8/IJCAI11-178
- [33] Piotr Ostropolski-Nalewaja and Sebastian Rudolph. 2024. The Sticky Path to Expressive Querying: Decidability of Navigational Queries under Existential Rules. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning, KR 2024, Hanoi, Vietnam, November 2-8, 2024*, Pierre Marquis, Magdalena Ortiz, and Maurice Pagnucco (Eds.). doi:10.24963/KR.2024/54
- [34] Terence Parr. 2012. ANTLR (ANother Tool for Language Recognition). <https://www.antlr.org/>. Accessed: 2026-05-21.
- [35] Ognjen Savkovic and Diego Calvanese. 2012. Introducing Datatypes in DL-Lite. In *ECAI 2012 - 20th European Conference on Artificial Intelligence. Including Prestigious Applications of Artificial Intelligence (PAIS-2012) System Demonstrations Track, Montpellier, France, August 27-31, 2012 (Frontiers in Artificial Intelligence and Applications)*. IOS Press, 720–725. doi:10.3233/978-1-61499-098-7-720
- [36] Michael J. Simons and Neo4j Contributors. 2020. The Neo4j Cypher-DSL. <https://github.com/neo4j/cypher-dsl>. Accessed: 2026-05-21.
- [37] Giorgio Stefanoni, Boris Motik, Markus Krötzsch, and Sebastian Rudolph. 2014. The Complexity of Answering Conjunctive and Navigational Queries over OWL 2 EL Knowledge Bases. *J. Artif. Intell. Res.* 51 (2014), 645–705. doi:10.1613/JAIR.4457
- [38] Andreas Steigmiller, Thorsten Liebig, and Birte Glimm. 2014. Konclude: System description. *J. Web Semant.* 27–28 (2014), 78–85. doi:10.1016/J.WEBSEM.2014.06.003
- [39] W3C OWL Working Group. 2012. OWL 2 Web Ontology Language Document Overview (Second Edition). <https://www.w3.org/TR/owl2-overview/>. W3C Recommendation. Accessed: 2026-05-21.
- [40] Guohui Xiao, Diego Calvanese, Roman Kontchakov, Domenico Lembo, Antonella Poggi, Riccardo Rosati, and Michael Zakharyashev. 2018. Ontology-Based Data Access: A Survey. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI 2018, July 13-19, 2018, Stockholm, Sweden*, Jérôme Lang (Ed.). ijcai.org, 5511–5519. doi:10.24963/IJCAI.2018/777

Received 23 May 2026; revised 7 August 2026; accepted 20 August 2026

A Proofs for Section 3 (Rewriting N2RPQs)

In order to prove Lemma 3, we recall the following standard fact on canonical model homomorphism from the Description Logic literature.

FACT 1. Let $\mathcal{T}$ be a TBox and $\mathcal{A}$ an ABox. Then, suppose that two nodes o_1 and o_2 are such that $o_1 \in A^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$ iff $o_2 \in A^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$ for every concept name A . Then, the two submodels $\mathcal{I}_{\mathcal{T},\mathcal{A}}$ rooted at o_1 and $\mathcal{I}_{\mathcal{T},\mathcal{A}}$ rooted at o_2 are isomorphic.

LEMMA 3. Let $\mathcal{T}$ be a DL-Lite_{PG} TBox and α be an N2RPE. Then, for every PG $\mathcal{A}$ and every pair of nodes o_s, o_f from $\mathcal{A}$, it holds that $(o_s, o_f) \in \alpha^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$ iff $(o_s, o_f) \in \text{skipAnon}^{\mathcal{T}}(\alpha)^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$.

PROOF. Consider an DL-Lite_{PG} TBox $\mathcal{T}$, a PG $\mathcal{A} = (N, E, \text{label}, \text{prop})$ and an N2RPE $\alpha = \langle Q, \Sigma_{PG}, S, \delta, F \rangle$ of nesting depth k . ($\Rightarrow$) For the first direction, we show the claim by induction on the nesting depth k of α . In the following we prove that, whenever $(o_s, o_f) \in \alpha^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$, then $(o_s, o_f) \in \text{skipAnon}^{\mathcal{T}}(\alpha)^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$ for every pair of nodes o_s, o_f in $\mathcal{A}$. From $(o_s, o_f) \in \alpha^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$, it follows that there is a run (T, l) for α on $\mathcal{I}_{\mathcal{T},\mathcal{A}}$ such that $l(o_s, s_0)$ and $l(o_f, s_f)$ with $s_0 \in S$ and $s_f \in F$.

Base Case. Suppose that $k = 0$, then T consists of a single path with the sequence of labels $(o_0, s_0)(o_1, s_1) \dots (o_m, s_m)$ from root to leaf. Note that $o_0 = o_s, o_m = o_f$ and $s_m = s_f$. We know from Definition 7 that for each $i \in \{1, \dots, m\}$ there is a transition $(s_{i-1}, \sigma_i, s_i) \in \delta$ such that one of the following cases holds (here we make use of the fact that there are no transitions with nested NFAs of the form $\langle \alpha_i \rangle$):

- $\sigma_i \in 2^{\text{RUT}^D}$ and either (1) or (2):
 - (1) $(o_{i-1}, o_i) \in (p \odot v?)^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$ for each data test $p \odot v? \in \sigma_i^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$
 - (2) $(o_{i-1}, o_i) \in r^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$ for each $r \in \sigma_i^{\mathcal{I}_{\mathcal{T},\mathcal{A}}} \cap \bar{R}$
- $\sigma_i = C?$, $o_{i-1} = o_i$ and $o_i \in C^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$
- $\sigma_i = p \odot v?$, $o_{i-1} = o_i$ and $o_i \in (p \odot v?)^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$

We can suppose without loss of generality that there is no $0 \leq i, i' \leq m$ such that $i \neq i', s_i = s_{i'}$, and $o_i = o_{i'}$ (this can always be ensured by deleting parts of the path). Now let $i_1 < \dots < i_p$ be all the indices i such that $o_i \in N$ of $\mathcal{A}$ (note that $i_1 = 0$ and $i_p = m$). Intuitively, $i_1, \dots, i_p$ are the indices of the "projection" of the run obtained by restricting it to individuals occurring in the ABox.

CLAIM 1 (CF. CLAIM 3 IN [BCOS14]). For every $1 < \ell \leq p$, one of the following holds:

- (1) $(o_{i_{\ell-1}}, o_{i_\ell}) \in \sigma_{i_\ell}^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$, or
- (2) $o_{i_\ell} = o_{i_{\ell-1}}$ and there exists a strictly anonymous tuple $(C, s_{i_{\ell-1}}, s_{i_\ell}, \emptyset) \in \text{SkipRel}(\alpha, \mathcal{T})$ such that $o_{i_{\ell-1}} \in C^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$.

Proof of Claim 1. For the case where $i_\ell = i_{\ell-1} + 1$ we immediately obtain $(o_{i_{\ell-1}}, o_{i_\ell}) \in \sigma_{i_\ell}^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$. Further, if $i_\ell \neq i_{\ell-1} + 1$, then $o_{i_{\ell-1}+1} \notin N$. From this we can infer that all the objects $o_{i_{\ell-1}+1}, \dots, o_{i_\ell-1}$ must be descendants of $o_{i_\ell} = o_{i_{\ell-1}}$ in $\mathcal{I}_{\mathcal{T},\mathcal{A}}$. Now, let $\mathcal{J}$ be the canonical model of $\mathcal{T}$ and $\{C(a)\}$. Then, by Fact 1 there is a homomorphism h from $\mathcal{I}_{\mathcal{T},\mathcal{A}}$ to $\mathcal{J}$ with $h(o_{i_{\ell-1}}) = a$. Consider the labelled tree (T', l') obtained from (T, l) by (1) making the node labelled $(o_{i_{\ell-1}}, s_{i_{\ell-1}})$ the root and the node labelled (o_{i_ℓ}, s_{i_ℓ}) the unique leaf, and (2) replacing every node label (o', s') by $(h(o'), s')$. It holds that (T', l') is a partial run of α on $\mathcal{J}$. Hence, by Definition 14, $(C, s_{i_{\ell-1}}, s_{i_\ell}, \emptyset) \in \text{SkipRel}(\alpha, \mathcal{T})$ and since $o_{i_{\ell-1}+1} \notin N$ it follows that the tuple is strictly anonymous. (end proof of Claim 1)

To conclude the base case for the main claim, we can simply construct a run (T, l) for $\text{skipAnon}^{\mathcal{T}}(\alpha)^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$, by combining all (o_{i_ℓ}, s_{i_ℓ}) from Claim 1. From the construction of $\text{skipAnon}^{\mathcal{T}}(\alpha)$ in Definition 17 it follows that $(o_s, o_f) \in \text{skipAnon}^{\mathcal{T}}(\alpha)^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$.

Induction Hypothesis. Assume that if $(o_s, o_f) \in \alpha^{\mathcal{I}_{\mathcal{T},\mathcal{A}}}$, then $(o_s, o_f) \in \text{skipAnon}^{\mathcal{T}}(\alpha)^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$ holds for α of nesting depth k .

Induction step. Let us now consider the case where α has nesting depth $k + 1$. Let $v_0, \dots, v_m$ be the sequence of nodes in T that begins with the root node labelled (o_s, s_0) and ends with the unique leaf node labelled (o_f, s_f) . In the following we use (o_ℓ, s_ℓ) for the label of node n_ℓ . We can again assume w.l.o.g. that (T, l) is minimal in the sense that there are no distinct positions $0 \leq i, i' \leq m$ such that n_i and $n_{i'}$ has the same label. Let $i_1 < \dots < i_p$ be all of the indices i such that $o_i \in N$ and $i_m = o_f \in N$.

CLAIM 2 (CF. CLAIM 6 IN [BCOS14]). For every $1 < \ell \leq p$, one of the following holds:

- (1) there is some α_j of nesting depth $\leq k + 1$ with $(s_{i_{\ell-1}}, \sigma, s_{i_\ell}) \in \delta_j$ such that $(o_{i_{\ell-1}}, o_{i_\ell}) \in \sigma_{i_\ell}^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$, or
- (2) $o_{i_\ell} = o_{i_{\ell-1}}$ and there exists a strictly anonymous tuple $(C, s_{i_{\ell-1}}, s_{i_\ell}, \Gamma) \in \text{skipRel}(\alpha, \mathcal{T})$ such that $o_{i_{\ell-1}} \in C^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$

Proof of Claim 2. From Definition 7 there are three possible cases:

- (a) n_{i_ℓ} is the unique child of node $n_{i_{\ell-1}}$
- (b) n_{i_ℓ} is one of two children of node $n_{i_{\ell-1}}$
- (c) node n_{i_ℓ} is a descendant, but not a child of $n_{i_{\ell-1}}$

First consider case (a), then by Definition 7 there must be a transition $(s_{i_{\ell-1}}, \sigma, s_{i_\ell}) \in \delta_j$ such that $(o_{i_{\ell-1}}, o_{i_\ell}) \in \sigma_{i_\ell}^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$, which satisfies the first statement of the claim. If case (b) holds, then by Definition 7 it follows that $(s_{i_{\ell-1}}, \langle \alpha' \rangle, s_{i_\ell}) \in \delta_j$, $o_{i_\ell} = o_{i_{\ell-1}}$, and the other child of $n_{i_{\ell-1}}$ has label $(o_{i_{\ell-1}}, u)$, with u the initial state of α' . The labelled subtree of (T, l) that is rooted at $(o_{i_{\ell-1}}, u)$ is a run for α' on $\mathcal{I}_{\mathcal{T},\mathcal{A}}$. Since the nesting depth of α' is $\leq k$, we can apply the induction hypothesis. Thus, the first statement holds, i.e., $(s_{i_{\ell-1}}, \sigma, s_{i_\ell}) \in \delta_j$ such that $(o_{i_{\ell-1}}, u) \in \sigma_{i_\ell}^{\mathcal{G}_{\mathcal{T},\mathcal{A}}}$. Finally, consider case (c) where n_{i_ℓ} is not a child of $n_{i_{\ell-1}}$, which means that $o_{i_{\ell-1}+1} \notin N$ of $\mathcal{A}$. From this we can infer that all nodes $o_{i_{\ell-1}+1}, \dots, o_{i_\ell-1}$ must be descendants of $o_{i_{\ell-1}}$ in $\mathcal{I}_{\mathcal{T},\mathcal{A}}$, and $o_{i_\ell} = o_{i_{\ell-1}}$. Let (T', l') be the partial run obtained from (T, l) by

1. making $n_{i_{\ell-1}}$ the new root node,
2. making v a leaf node by dropping all children, for every descendant v of $n_{i_{\ell-1}}$ such that $l(v) = (o_{i_{\ell-1}}, s)$ and such that there is no v' (different from v and $n_{i_{\ell-1}}$) with $l(v') = (o_{i_{\ell-1}}, s')$ and occurring along the path from $n_{i_{\ell-1}}$ to v .

Note that since (T, l) is a run, there are three types of leaf nodes in (T', l') :

- node n_{i_ℓ} with label $(o_{i_{\ell-1}}, s_{i_\ell})$ (on the main path)
- nodes with labels of the form $(o_{i_{\ell-1}}, s)$
- nodes with labels of the form (o, s_f) with $s_f \in F_j$ for some α_j

(Observe that every label (o, s) that appears in (T', l') is an individual in the subtree in the anonymous part of $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ rooted at $o_{i_{\ell-1}}$.) Now let $\mathcal{J}$ be the canonical model of $\mathcal{T}$ and $C(d)$, then it follows from Fact 1 there exists a homomorphism h between $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ to $\mathcal{J}$ with $h(o_{i_{\ell-1}}) = d$. Consider the labelled tree (T'', l'') obtained from (T', l') by replacing every node label (o, s) by $(h(o), s)$. Using the fact that h is a homomorphism with $h(o_{i_{\ell-1}}) = d$ and the above description of leaf nodes in (T', l') , one can show that (T'', l'') is a partial run of α on $\mathcal{J}$ that satisfies:

- the root of T'' is labelled $(d, s_{i_{\ell-1}})$
- there is a leaf node labelled (d, s_{i_ℓ})
- for every leaf node v with $l(v) = (o, s) \neq (d, s_{i_\ell})$, either $s \in F_j$ or $o = d$

Since this partial run satisfies the conditions in Definition 14 the tuple $(C, s_{i_{\ell-1}}, s_{i_\ell}, \Gamma)$ belongs to $\text{skipRel}(\alpha, \mathcal{T})$, if we let Γ contain all those states u such that there is a leaf node (d, u) with $u \neq s_{i_\ell}$. Since we have that $o_{i_{\ell-1}+1} \notin N$, the tuple is strictly anonymous and the second statement of the claim holds. (*end of proof Claim 2*)

To conclude the proof for the first direction, we construct a run (T, l) for $\text{skipAnon}^{\mathcal{T}}(\alpha)$ in $\mathcal{G}_{\mathcal{T}, \mathcal{A}}$ in a similar fashion as for the base case by combining all labels (o_{i_ℓ}, s_{i_ℓ}) for $1 < \ell < p$. For tuples $(C, s_{i_{\ell-1}}, s_{i_\ell}, \Gamma) \in \text{skipRel}(\alpha, \mathcal{T})$ the run contains labels (o_{i_ℓ}, u) for each $u \in \Gamma$. By induction hypothesis there is a run (T_u, l_u) for α^u and by Definition 17 we can verify that (T, l) is a valid run for $\text{skipAnon}^{\mathcal{T}}(\alpha)$, thus $(o_s, o_f) \in \text{skipAnon}^{\mathcal{T}}(\alpha)^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$ holds.

($\Leftarrow$) The second direction we prove by induction on the nesting depth of $\text{skipAnon}^{\mathcal{T}}(\alpha)$. Suppose that $(o_s, o_f) \in \text{skipAnon}^{\mathcal{T}}(\alpha)^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$ for $\text{skipAnon}^{\mathcal{T}}(\alpha) = \langle Q^{\text{skip}}, \Sigma_{PG}, S^{\text{skip}}, \delta^{\text{skip}}, F^{\text{skip}} \rangle$, then we prove that $(o_s, o_f) \in \alpha^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$. From $(o_s, o_f) \in \text{skipAnon}^{\mathcal{T}}(\alpha)^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$ it follows that there is a run (T, l) for $\text{skipAnon}^{\mathcal{T}}(\alpha)$ on $\mathcal{G}_{\mathcal{T}, \mathcal{A}}$ such that $l(o_s, s_0)$ and $l(o_f, s_f)$ with $s \in S^{\text{skip}}$ and $s_f \in F^{\text{skip}}$.

Base Case. Consider the nesting depth of $\text{skipAnon}^{\mathcal{T}}(\alpha)$ to be $k = 0$. Then, T has only a single path with the sequence of labels $(o_0, s_0)(o_1, s_1) \dots (o_m, s_m)$ from root $o_0 = o_s$ to leaf $o_m = o_f$. It suffices to show the following claim.

CLAIM 3 (CF. CLAIM 2 IN [BCOS14]). *For every $0 \leq i \leq m$, it holds that $(o_i, o_m) \in \alpha_{|s_i}^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$.*

Proof of Claim 3. The proof is by induction on i . The **base case** is when $i = m$, in which case $s_i = s_m$ and the claim trivially holds since s_m is a final state. Now suppose that the claim holds for i such that $g < i \leq m$ (**induction hypothesis**). Consider the induction step with $i - 1 = g$, we know by Definition 17 that (o_g, s_g) must satisfy one of the following conditions: (i) there is a transition $(s_g, \sigma, s_i) \in \delta$, or (ii) there is a transition (s_g, C, s_i) such that there is a tuple $(C, s_g, s_i, \emptyset) \in \text{skipRel}(\alpha, \mathcal{T})$. If (i) holds, then $(o_g, o_i) \in \sigma^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$ by construction of the canonical model Definition 13. By induction hypothesis there is a (T_i, l_i) run, which is a (o_i, o_m) -run for $\alpha_{|s_i}$ on $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$. Now, let (T', l') be the labelled tree obtained by creating a new node labelled (o_g, s_g) and making it the leaf of (T', l') . It is easy to verify that (T', l') is a run of $\alpha_{|s_g}$ on $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$. Next consider the case (ii) where $(C, s_g, s_i, \emptyset) \in \text{skipRel}(\alpha, \mathcal{T})$, from which follows that $o_g = o_i$ and $o_g \in C^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$. From Definition 14 we can infer that there is a partial run (T_C, l_C) of α in the canonical model of $(\mathcal{T}, \{C(a)\})$ that satisfies the conditions below:

- the root of T_C is labelled (a, s_g)
- there is a unique leaf node v with $l_C(v) = (a, s_i)$

Since $o_g \in C^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$ it follows from Fact 1 that there is a homomorphism h from the canonical model of $(\mathcal{T}, \{C(a)\})$ to $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ with $h(a) = o_g$. We obtain (T'_C, l'_C) by replacing every label (o, s) in T_C by $(h(o), s)$. Using the fact that h is a homomorphism, one can show that (T'_C, l'_C) defines a partial run of α in $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ that satisfies the following conditions:

- the root of T'_C is labelled (o_g, s_g)
- there is a unique leaf node v with $l'_C(v) = (o, s_i)$

Let (T, l) be the labelled tree obtained from (T'_C, l'_C) by replacing the unique leaf node labelled (o_g, s_i) by the tree (T_i, l_i) . It follows from the components that (T, l) is a (o_g, o_m) -run of $\alpha_{|s_g}$ on $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$. (*end of proof of Claim 3*)

Since s_0 is an initial state of α , the statement can be generalized and it follows that $(o_0, o_m) \in \alpha^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$.

Induction Hypothesis. Assume that if $(o_s, o_f) \in \text{skipAnon}^{\mathcal{T}}(\alpha)^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$, then $(o_s, o_f) \in \alpha^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$ holds for $\text{skipAnon}^{\mathcal{T}}(\alpha)$ of nesting depth k .

Induction Step. Suppose that $(o_0, s_0)(o_1, s_1) \dots (o_m, s_m)$ is the sequence of the run $(o_s, o_f) \in \alpha^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$ such that $o_s = o_0$, $o_f = o_m$ and $s_0, \dots, s_m \in Q$, i.e., states of the automaton with the highest nesting depth.

CLAIM 4 (CF. CLAIM 4 IN [BCOS14]). *For every $0 \leq i \leq m$, it holds that $(o_i, o_m) \in \alpha_{|s_i}^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$.*

Proof of Claim 4. The proof is by induction on i . The base case is when $i = m$, in which we consider a tree with a single node $i = m$ and the claim is trivially satisfied since s_m is a final state. Now suppose that the claim holds for i such that $g < i \leq m$. Consider the case

where $i - 1 = g$, then the pair (o_i, s_i) satisfies one of the conditions, either: (i) there is a transition $(s_g, \sigma, s_i) \in \delta$ or (ii) there is a tuple $(C, s_g, s_i, \{s_1^\Gamma, \dots, s_n^\Gamma\}) \in \text{skipRel}(\alpha, \mathcal{T})$ and there are corresponding transitions $(s_g, C, s_1'), (s_1', \langle \alpha_{|s_1^\Gamma}^1 \rangle, s_2'), \dots, (s_n', \langle \alpha_{|s_n^\Gamma}^n \rangle)$ in $\text{skipAnon}^\mathcal{T}(\alpha)$. If (i) holds we can immediately construct a (o_g, o_m) -run of $\alpha_{|s_g}$ on $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ in the same manner as in the base case. Next assume that case (ii) holds for (o_g, s_g) . Then, $o_g = o_i$ and there exists a tuple (C, s_g, s_i, Γ) such that $o_g \in C^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$. By Definition 14 the existence of (C, s_g, s_i, Γ) implies that there is a partial run (T_C, l_C) of α in the canonical model of $(\mathcal{T}, \{C(a)\})$ that satisfies the following conditions:

- the root of T_C has label (a, s_g)
- there is a leaf node v with $l(v) = (a, s_i)$
- for every leaf node v with $l_C(v) = (o, s) \neq (a, s_i)$, either $s \in F_j$, or $o = a$ and $s \in \Gamma$

Since $o_g \in C^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$, it follows from Fact 1 that there is a homomorphism h from the canonical model of $(T, \{C(a)\})$ to $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ with $h(a) = o_g$. Let (T', l') be obtained by replacing every label (o, s) in T_C by $(h(o), s)$. Since h is a homomorphism, one can show that (T'_C, l'_C) defines a partial run of α in $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$ that satisfies the following conditions:

- the root of T'_C is labelled (o_g, s_g)
- there is a leaf node v with $l'_C(v) = (o_g, s_i)$
- for every leaf node v with $l'_C(v) = (o, s) \neq (o_g, s_i)$, either $s \in F_j$, or $o = o_g$ and $s \in \Gamma$.

Next we consider the states in Γ , by induction hypothesis we can find for each $s_j \in \Gamma$ a run $(T_{s_j}^\Gamma, l_{s_j}^\Gamma)$ for $\alpha_{|s_j}^\Gamma$ on $\mathcal{I}_{\mathcal{T}, \mathcal{A}}$. Finally, we can conclude the claim by applying the induction hypothesis and the fact that $c_g = c_i$. Thus, we can construct a run (T_g, l_g) such that $(o_g, o_m) \in \alpha^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$ from (T'_C, l'_C) :

- replace (unique) leaf node labelled (o_g, s_i) by the tree (T_i, l_i)
- replace each leaf node labelled (o_g, s_j) for $s_j \in \Gamma$ by the tree $(T_{s_j}^\Gamma, l_{s_j}^\Gamma)$

(end proof of Claim 4)

From Claim 4) it follows that $(o_s, o_f) \in \alpha_{|o_s}^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$, since o_s is an initial state of α , we can conclude that $(o_s, o_f) \in \alpha^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}$. $\square$

LEMMA 4. Let $\mathcal{T}$ be a DL-Lite_{PG} TBox and α be an N2RPE. Then, for every PG $\mathcal{A}$ and every pair of nodes o_s, o_f from $\mathcal{A}$, it holds that $(o_s, o_f) \in \alpha^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$ iff $(o_s, o_f) \in \text{skipCore}^\mathcal{T}(\alpha)^\mathcal{A}$.

PROOF. Let $\mathcal{T}_{\text{core}}^*$ be the restriction of $\mathcal{T}^*$ to the axioms that do not have existential concepts on the right, and for readability, for a given $\mathcal{A}$, we denote by $\mathcal{A}^*$ the canonical model $\mathcal{I}_{\mathcal{T}_{\text{core}}^*, \mathcal{A}}$. One can prove in the usual way (see, e.g., [EOS⁺12]) that $\mathcal{G}_{\mathcal{T}, \mathcal{A}} = \mathcal{A}^*$. We can thus show that, for every α :

$$\alpha^{\mathcal{A}^*} = (\text{skipCore}^\mathcal{T}(\alpha))^\mathcal{A}.$$

Given the one-to-one correspondence between the rules (Ch1) to (Ch5), which add facts to $\mathcal{A}^*$ and the rules (R1) to (R5), which add transitions to $\text{skipCore}^\mathcal{T}(\alpha)$, the equality is established by a simple rule-by-rule analysis in both directions.

For the ($\Leftarrow$) direction, assume that an n-NFA α_{i+1} was obtained by applying one of the rules (R1) to (R5) in Definition 19 to α_i . In the following we show for each of the rules that it holds if $(o_s, o_f) \in \alpha_{i+1}^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}(x, y)$, then $(o_s, o_f) \in \alpha_i^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}(x, y)$.

- (R1) Assume that $\alpha_{i+1}^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}(x, y)$ was obtained by applying the rule for $r \sqsubseteq p \in \mathcal{T}$ and $(s_j, P, s_{j+1}) \in \delta_i$ with $p \in P$ (or $p^- \in P$). Then, $\alpha_{i+1}^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}(x, y)$ contains the transition (s_j, R, s_{j+1}) with $R = (P \setminus \{p\}) \cup \{r\}$ (or $R = (P \setminus \{p^-\}) \cup \{r^-\}$). We assume that $(o_j, o_{j+1}) \in r^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$ (or $(o_{j+1}, o_j) \in r^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$) and from Definition 13 it follows that $(o_j, o_{j+1}) \in p^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$ (or $(o_{j+1}, o_j) \in p^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$).
- (R2), (R3) The proof for these rules works equivalent to (R1).
- (R4) Assume that $\alpha_{i+1}^{\mathcal{I}_{\mathcal{T}, \mathcal{A}}}(x, y)$ was obtained by the rule for $\exists r. \top \sqsubseteq B$ and $(s_j, B?, s_{j+1}) \in \delta_i$. Then, the function adds a transition with a nested NFA $\langle \alpha_r \rangle$ that contains a single transition with r . By Definition 7 of a run and α_{i+1} it holds that there has to be an individual $o_j = o_{j+1}$ and a pair (o_j, o') , such that $(o_j, o') \in r^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$. From Definition 13 it follows that $o_j \in B^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$ and thus $(o_j, o_{j+1}) \in \alpha_i^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$.
- (R5) In this case α_{i+1} was obtained based on $T \sqsubseteq D \in \mathcal{T}^*$ and $(s_j, D?, s_{j+1}) \in \delta_i$. For each $t_\ell \in T$ there is a transition $(s'_\ell, t_\ell?, s'_{\ell+1}) \in \alpha_{i+1}$, where $s_j = s'_1, s'_{k+1} = s_j$ and $s'_2, \dots, s'_k$ are fresh states, i.e., $s'_2, \dots, s'_k \notin S_i$. By Definition 7 and $(s'_\ell, t_\ell?, s'_{\ell+1}) \in \alpha_{i+1}$, it follows that $o_j = o_{j+1}$ and $o_j \in T^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$. Further, by the construction of the canonical model item (Ch5) we can infer that $o_j \in D^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$.

($\Rightarrow$) In order to show completeness we first introduce the notion of degree, that comes from the canonical model construction: We denote individuals or pair of individuals in the extension of concepts and roles as *facts*, i.e., $v \in B^{\mathcal{G}_j}$ or $(v, w) \in r^{\mathcal{G}_j}$. A fact that occurs for the first time in $\mathcal{G}_j$ has degree j , and a set of facts gets the sum of the degrees of its elements. The degree of sequence $o_s o_{\sigma_1} o_1 \dots o_n o_n o_f$ witnessing $(o_s, o_f) \in \alpha^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}(x, y)$ is the minimal degree of a set of facts that is sufficient to witness all a run in Definition 7, including the nested automata. Note that there may be more than one way to witness not only the automata, but even for a fixed (top level) sequence, we may have choices, but we don't care. We set the degree to be the smallest, and then every added transition will allow us to witness the same sequence but with a strictly smaller degree. Having the notion of degree for a pair in place we show by induction that in each rewriting step we are "moving up" the part of the canonical model (decreasing the degree) that is induced by the axioms in $\mathcal{T}$. In the following we show

that for each of the rules in Definition 13 there is a corresponding rule in Definition 17 producing a n-NFA α_{i+1} from α_i such that the degree of the accepted sequence is strictly smaller.

- (Ch1) Consider $r \sqsubseteq p \in \mathcal{T}$ and $(o_k, o_{k+1}) \in r^{\mathcal{G}_j}$, $(o_k, o_{k+1}) \notin p^{\mathcal{G}_j}$ and $(o_k, o_{k+1}) \in p^{\mathcal{G}_{j+1}}$ by construction of the canonical model. Consider the sequence $(o_s \sigma_1 o_1 \dots o_k r o_{k+1} \dots o_n \sigma_f o_f)$ such that $(o_s, o_f) \in \alpha_i^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$. In the rewriting we apply (R1) to obtain α_{i+1} from α_i , where we add for each transition (s, P, s') with $p \in P$ (or $p^- \in P$) a new transition (s, R, s') such that $R = (P \setminus \{p\}) \cup \{r\}$ (or $R = (P \setminus \{p^-\}) \cup \{r^-\}$), thus $(o_1 \dots o_k r o_{k+1} \dots o_n)$ will be accepted by $\alpha_{i+1}^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$, for which the degree is strictly smaller then for $(o_s \sigma_1 o_1 \dots o_k p o_k \dots o_n \sigma_f o_f)$.
- (Ch2),(Ch3) For $T \sqsubseteq r' \in \mathcal{T}$ the skipping rule (R2) and for $A \sqsubseteq B \in \mathcal{T}$ (R3) applies. In both cases the reasoning is similar to (Ch1).
- (Ch4) Given $\exists r. \top \sqsubseteq B \in \mathcal{T}$ and by construction of the canonical model some individual $o_k \in (\exists r. \top)^{\mathcal{G}_j}$, $o_k \notin B^{\mathcal{G}_j}$ $o_k \in B^{T_{j+1}}$. Consider a sequence of the form $(o_s \sigma_1 o_1 \dots o_k B o_k \dots o_n \sigma_f o_f)$ such that $(o_s, o_f) \in \alpha_i^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$. In skipping we apply rule (R4) to obtain α_{i+1} and add for each $(s, B?, s') \in \delta$ the transition $(s, \langle \alpha_r \rangle, s')$ to δ , where α_r is a fresh two-state NFA with a single transition $(s_0^1, \{r\}, s_f^1)$. Since $o_k \in (\exists r. \top)^{\mathcal{G}_j}$ there has to be some sequence $(o_k r o'_k)$, such that $(o_k, o'_k) \in \alpha_r^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$ and we can construct a run with strictly smaller degree such that $(o_s, o_f) \in \alpha_{i+1}^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$.
- (Ch5) In this case that $T \sqsubseteq B \in \mathcal{T}$, $o_k \in T_j^{\mathcal{G}}$ and $o_k \notin B_j^{\mathcal{G}}$ triggered the rule. Consider again the sequence $(o_s \sigma_1 o_1 \dots o_k B o_k \dots o_n \sigma_f o_f)$ with $(o_s, o_f) \in \alpha_i^{\mathcal{G}_{\mathcal{T}, \mathcal{A}}}$. Based on $T \sqsubseteq B \in \mathcal{T}$, $o_k \in T_j^{\mathcal{G}}$ the skipping function applies rule (R5). It follows that for $(s, B?, s') \in \delta$ we add for each $t_\ell \in T$ a transition $(s_\ell, t_\ell?, s_{\ell+1})$, where $s = s_1$ and $s_{j+1} = s'$, and $s'_2, \dots, s'_k$ are fresh states. Since $o_k \in T^{\mathcal{G}_j}$, it holds that $o_k \in t_\ell^{\mathcal{G}_j}$ for each of the $t_\ell \in T$ and the degree of the sequence from o_s to o_f is strictly smaller.
- (Ch6),(Ch7) Since we only consider $\mathcal{T}_{core}^*$ these rules do not apply (and are covered by Definition 17). □

References for the Appendix

- [BCOS14] Meghyn Bienvenu, Diego Calvanese, Magdalena Ortiz, and Mantas Simkus. Nested regular path queries in description logics. In Chitta Baral, Giuseppe De Giacomo, and Thomas Eiter, editors, *Principles of Knowledge Representation and Reasoning: Proceedings of the Fourteenth International Conference, KR 2014, Vienna, Austria, July 20-24, 2014*. AAAI Press, 2014.
- [EOS⁺12] Thomas Eiter, Magdalena Ortiz, Mantas Simkus, Trung-Kien Tran, and Guohui Xiao. Query rewriting for Horn-SHIQ plus rules. In Jörg Hoffmann and Bart Selman, editors, *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence, July 22-26, 2012, Toronto, Ontario, Canada*, pages 726–733. AAAI Press, 2012.